

BỘ GIAO THÔNG VẬN TẢI
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM
VIỆN ĐÀO TẠO CHẤT LƯỢNG CAO

—o0o—



BÁO CÁO: THUẬT TOÁN ĐỒ THỊ

HỌC PHẦN: CẤU TRÚC RỜI RẠC

GIÁO VIÊN HƯỚNG DẪN: TĂNG LÊ NGỌC GIA HUY

TP. Hồ Chí Minh, tháng 9 / 2026

STT	Họ và tên	Mã số sinh viên	Phần trăm đóng góp	Nhận xét
1	Nguyễn Minh Khánh	087206013555	85%	Trễ deadline
2	Trần Lê Đông Nghị	079306041859	100%	Hoàn thành đầy đủ
3	Phạm Minh Nhật	068206010555	100%	Hoàn thành đầy đủ
4	Hồ Ngọc Gia Hân	079305030460	100%	Hoàn thành đầy đủ
5	Nguyễn Võ Gia Anh	052207001422	90%	Không tham gia họp
6	Phạm Quốc Bảo	082207009909	0%	Không tham gia

LỜI CẢM ƠN

Lời đầu tiên, em xin bày tỏ lòng biết ơn sâu sắc và chân thành nhất đến Thầy Tăng Lê Ngọc Gia Huy - Giảng viên hướng dẫn, người đã luôn tận tâm theo sát, định hướng phương pháp tiếp cận và truyền đạt những kiến thức chuyên môn quý báu giúp em hoàn thành tốt đề tài này.

Sự chỉ dẫn khoa học, tận tình cùng những đóng góp sâu sắc của Thầy trong suốt quá trình phân tích thuật toán, thiết kế hệ thống và giải quyết các bài toán kỹ thuật là nguồn hỗ trợ then chốt giúp ứng dụng đạt được kết quả như kỳ vọng.

Mặc dù đã nỗ lực hoàn thiện đề tài một cách cẩn chu nhất, song bài báo cáo chắc chắn khó tránh khỏi những điểm hạn chế. Em rất mong tiếp tục nhận được những ý kiến đóng góp, chỉ bảo quý báu từ Thầy để hoàn thiện hơn nữa kiến thức và kỹ năng của bản thân trong tương lai.

Kính chúc Thầy Tăng Lê Ngọc Gia Huy luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp giảng dạy và nghiên cứu khoa học!

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN	1
1.1. Lý do chọn đề tài	1
1.2. Mục tiêu nghiên cứu	2
1.3. Phạm vi và giới hạn đề tài	2
1.3.1. Phạm vi nghiên cứu	2
1.3.2. Giới hạn đề tài	3
1.4. Phương pháp nghiên cứu	3
1.4.1. Nghiên cứu tài liệu	3
1.4.2. Phân tích và mô tả thuật toán	4
1.4.3. Cài đặt và kiểm thử	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	5
2.1. Lý thuyết đồ thị	5
2.1.1. Định nghĩa đồ thị	5
2.1.2. Phân loại đồ thị	6
2.1.3. Các khái niệm và định lý cơ bản	8
2.2. Cấu trúc dữ liệu biểu diễn đồ thị	9
2.2.1. Danh sách kề (Adjacency List)	9
2.2.2. Định dạng file dữ liệu đầu vào (JSON)	10
2.2.3. Đồ thị thặng dư (Residual Graph) trong Ford-Fulkerson	11
2.3. Tổng quan các nhóm bài toán và thuật toán	11
2.3.1. Nhóm duyệt đồ thị (traversal/)	11
2.3.2. Nhóm đường đi ngắn nhất (shortest_path/)	12
2.3.3. Nhóm cây khung nhỏ nhất (minimum_spanning_tree/)	14
2.3.4. Nhóm thuật toán nâng cao (advanced/)	15
2.4. Kiến trúc mã nguồn tổng thể của dự án	17

CHƯƠNG 3: PHÂN TÍCH CÁC THUẬT TOÁN	19
3.1. Thuật toán BFS (Breadth-First Search)	19
3.1.1. Ý tưởng	19
3.1.2. Code	19
3.1.3. Phân tích độ phức tạp	19
3.1.4. Ưu và nhược điểm	20
3.2. Thuật toán DFS (Depth-First Search)	20
3.2.1. Ý tưởng	20
3.2.2. Giả mã	20
3.2.3. Phân tích độ phức tạp	21
3.2.4. Ưu và nhược điểm	21
3.3. Thuật toán Dijkstra	22
3.3.1. Ý tưởng	22
3.3.2. Code	22
3.3.3. Phân tích độ phức tạp	23
3.3.4. Ưu và nhược điểm	23
3.4. Thuật toán Bellman-Ford	25
3.4.1. Ý tưởng	25
3.4.2. Code	25
3.4.3. Phân tích độ phức tạp	26
3.4.4. So sánh Dijkstra và Bellman-Ford	26
3.5. Thuật toán Kruskal	27
3.5.1. Ý tưởng	28
3.5.2. Code	28
3.5.3. Phân tích độ phức tạp	28
3.5.4. Ưu và nhược điểm	29

3.6. Thuật toán Prim	31
3.6.1. Ý tưởng.....	31
3.6.2. Code.....	31
3.6.3. Phân tích độ phức tạp.....	32
3.6.4. So sánh Kruskal và Prim.....	34
3.7. Thuật toán Fleury	35
3.7.1. Ý tưởng.....	35
3.7.2. Điều kiện tiên quyết.....	35
3.7.3. Code.....	35
3.7.6. Ưu và nhược điểm.....	37
3.8. Thuật toán Hierholzer	37
3.8.1. Ý tưởng.....	37
3.8.2. Code.....	37
3.8.3. Phân tích độ phức tạp.....	38
3.8.4. So sánh Fleury và Hierholzer	39
3.9. Thuật toán Ford-Fulkerson (Edmonds-Karp)	39
3.9.1. Ý tưởng.....	39
3.9.2. Code.....	39
3.9.3. Phân tích độ phức tạp.....	40
3.9.4. Ưu và nhược điểm.....	41
3.10. Ứng dụng thực tế – Tìm tuyến xe buýt ngắn nhất	43
3.10.1. Ý tưởng.....	43
3.10.2. Giải thích cấu trúc chương trình.....	43
3.10.3. Phân tích từng phần code.....	44
3.10.5. Phân tích độ phức tạp.....	49
3.10.6. Ưu và nhược điểm	49

CHƯƠNG 4: CÀI ĐẶT VÀ THỰC NGHIỆM	51
4.1. Môi trường cài đặt	51
4.1.1. Ngôn ngữ và phiên bản	51
4.1.2. Cài đặt và khởi chạy	51
4.2. Kiến trúc giao diện chương trình	52
4.2.1. Tổng quan kiến trúc UI	52
4.2.2. Mô tả các tab chức năng	53
4.3. Bộ dữ liệu kiểm thử	54
4.4. Kết quả thực nghiệm	56
4.4.1. Thực nghiệm 1 - BFS và DFS trên 1.json (5 đỉnh)	56
4.4.2. Thực nghiệm 2 - Dijkstra và Bellman-Ford trên UndirectedGraph-11.json (11 đỉnh)	56
4.4.3. Thực nghiệm 3 - Bellman-Ford phát hiện chu trình âm trên (Directed)–Trọng số âm có Chu trình âm.json	57
4.4.4. Thực nghiệm 4 - Dijkstra trên (Directed)– Trọng số thập phân.json	58
4.4.5. Thực nghiệm 5 - Kruskal và Prim trên UndirectedGraph- 11.json	59
4.4.6. Thực nghiệm 6 - Fleury và Hierholzer trên (Undirected)– Đồ thị Euler.json	60
4.4.7. Thực nghiệm 7 — Ford-Fulkerson trên mô phỏng mạng.json	62
4.5. Nhận xét và đánh giá tổng hợp	65
4.5.1. Về tính đúng đắn	65
4.5.2. Về hiệu suất	66
4.5.3. Về giao diện trực quan	67
4.6. Hạn chế còn tồn tại	67

CHƯƠNG 5: KẾT LUẬN	69
5.1. Tổng kết kết quả đạt được	69
<i>5.1.1. Về lý thuyết</i>	69
<i>5.1.2. Về cài đặt</i>	69
<i>5.1.3. Về giao diện và thực nghiệm</i>	70
5.2. Những điều rút ra	71
<i>5.2.1. Về lựa chọn thuật toán</i>	71
<i>5.2.2. Về thiết kế phần mềm</i>	72
5.3. Hạn chế còn tồn tại	72
5.4. Lời kết.....	73

DANH MỤC HÌNH ẢNH

<i>Hình 2.1. Giao diện trực quan hóa đồ thị</i>	6
<i>Hình 2.2. Giao diện khởi tạo đồ thị</i>	8
<i>Hình 2.3. Giao diện hiển thị dữ liệu đồ thị</i>	10
<i>Hình 2.5. Giao diện thực hiện thuật toán BFS và DFS</i>	12
<i>Hình 2.6. Giao diện thuật toán Dijkstra và Bellman-Ford</i>	14
<i>Hình 2.8. Giao diện trực quan hóa thuật toán nâng cao</i>	15
<i>Hình 2.9. Giao diện thực hiện thuật toán Fleury</i>	16
<i>Hình 2.10. Giao diện thực hiện thuật toán Hierholzer</i>	16
<i>Hình 2.11. Giao diện thực hiện thuật toán Ford-Fulkerson</i>	17
<i>Hình 3.1. Mã nguồn thuật toán BFS</i>	19
<i>Hình 3.2. Mã nguồn thuật toán DFS</i>	21
<i>Hình 3.3. Mã nguồn thuật toán Dijkstra</i>	23
<i>Hình 3.4. Mã nguồn thuật toán Bellman-Ford</i>	26
<i>Hình 3.5. Mã nguồn thuật toán Kruskal</i>	28
<i>Hình 3.6. Mã nguồn thuật toán Prim</i>	32
<i>Hình 3.7. Mã nguồn thuật toán Fleury</i>	36
<i>Hình 13.8. Mã nguồn thuật toán Hierholzer</i>	38
<i>Hình 3.9. Mã nguồn thuật toán Ford-Fulkerson</i>	40
<i>Hình. Kiểm tra điều kiện trước khi chạy thuật toán</i>	47
<i>Hình. Lệnh gọi Dijkstra và lấy đường đi</i>	47
<i>Hình. Phần code tạo chi tiết từng chặng</i>	48

<i>Hình. Hiện thị và trực quan hóa kết quả.....</i>	<i>49</i>
---	-----------

DANH MỤC BẢNG BIỂU

<i>Bảng 2.1. Phân loại các loại đồ thị sử dụng trong dự án</i>	7
<i>Bảng 2.2. So sánh BFS và DFS</i>	12
<i>Bảng Tổng quan thuật toán Dijkstra và Bellman-Ford</i>	13
<i>Bảng Tổng quan thuật toán Kruskal và Prim</i>	15
<i>Bảng 3.1. Độ phức tạp của thuật toán BFS</i>	20
<i>Bảng 3.2. Ưu và nhược điểm của thuật toán BFS</i>	20
<i>Bảng 3.4. Độ phức tạp của thuật toán DFS</i>	21
<i>Bảng 3.5. Ưu và nhược điểm của thuật toán DFS</i>	22
<i>Bảng Độ phức tạp của thuật toán Dijkstra</i>	23
<i>Bảng Ưu và nhược điểm của thuật toán Dijkstra</i>	24
<i>Bảng Độ phức tạp của thuật toán Bellman-Ford</i>	26
<i>Bảng So sánh thuật toán Dijkstra và Bellman-Ford</i>	27
<i>Bảng Độ phức tạp của thuật toán Kruskal</i>	29
<i>Bảng Ưu và nhược điểm của thuật toán Kruskal</i>	29
<i>Bảng Độ phức tạp của thuật toán Prim</i>	33
<i>Bảng So sánh thuật toán Kruskal và Prim</i>	35
<i>Bảng Độ phức tạp của thuật toán Fleury</i>	37
<i>Bảng Ưu và nhược điểm của thuật toán Fleury</i>	37
<i>Bảng Độ phức tạp của thuật toán Ford-Fulkerson</i>	41
<i>Bảng Ưu và nhược điểm của thuật toán Ford-Fulkerson</i>	41
<i>Bảng. Độ phức tạp của chức năng tìm tuyến xe buýt</i>	49
<i>Bảng. Ưu và nhược điểm của ứng dụng tìm tuyến xe buýt</i>	50

<i>Bảng Môi trường và công cụ phát triển hệ thống</i>	51
<i>Bảng Chức năng các tab của chương trình</i>	54
<i>Bảng Danh sách các bộ dữ liệu kiểm thử</i>	55
<i>Bảng Kết quả thực nghiệm BFS và DFS trên 1.json</i>	56
<i>Bảng 4.2. Kết quả thực nghiệm Dijkstra và Bellman-Ford</i>	57
<i>Bảng 4.4. Kết quả Dijkstra trên đồ thị có trọng số thập phân</i>	58
<i>Bảng 4.5. Kết quả thực nghiệm Kruskal và Prim</i>	60
<i>Bảng 4.6. Kết quả thực nghiệm Fleury và Hierholzer</i>	61
<i>Bảng 4.7. Kết quả thực nghiệm Ford-Fulkerson</i>	63
<i>Bảng Phân bố luồng và tỷ lệ sử dụng trên các liên kết</i>	64
<i>Bảng So sánh hiệu suất của các thuật toán</i>	67
<i>Bảng Các hạn chế của hệ thống và hướng khắc phục</i>	68
<i>Bảng Các module và thuật toán đã cài đặt</i>	69
<i>Bảng Tổng hợp các hạn chế còn tồn tại của đề tài</i>	73

CHƯƠNG 1: TỔNG QUAN

1.1. Lý do chọn đề tài

Trong thời đại công nghệ số phát triển vượt bậc, các bài toán liên quan đến mạng lưới và kết nối ngày càng trở nên phổ biến và quan trọng. Từ hệ thống định tuyến mạng máy tính, các ứng dụng tìm đường trong bản đồ số như Google Maps, cho đến các nền tảng mạng xã hội lớn như Facebook, Twitter - tất cả đều ứng dụng lý thuyết đồ thị và các thuật toán đồ thị ở cốt lõi của mình.

Đồ thị (Graph) là một trong những cấu trúc dữ liệu trừu tượng và linh hoạt nhất trong khoa học máy tính, cho phép mô hình hóa các mối quan hệ phức tạp giữa các thực thể. Các thuật toán đồ thị đóng vai trò nền tảng trong nhiều lĩnh vực: tìm đường đi ngắn nhất (Dijkstra, Bellman-Ford), xây dựng mạng lưới tối ưu (Kruskal, Prim), tìm luồng cực đại trong mạng (Ford-Fulkerson), cho đến tìm chu trình Euler trong các bài toán thực tế (Fleury, Hierholzer).

Tuy nhiên, mỗi thuật toán lại có những đặc điểm, ưu điểm và giới hạn riêng. Việc lựa chọn thuật toán phù hợp với từng bài toán cụ thể đòi hỏi lập trình viên phải nắm vững nguyên lý hoạt động cũng như độ phức tạp thời gian và không gian của từng thuật toán. Đây chính là động lực thúc đẩy nhóm nghiên cứu, cài đặt và trực quan hóa một tập hợp các thuật toán đồ thị tiêu biểu, từ đó có cái nhìn so sánh toàn diện và khả năng áp dụng thực tiễn.

Đề tài "Thuật Toán Đồ Thị" được chọn không chỉ vì tính ứng dụng cao mà còn vì đây là nền tảng kiến thức quan trọng đối với bất kỳ kỹ sư phần mềm nào. Thông qua việc nghiên cứu, cài đặt và phân tích các thuật toán bằng ngôn ngữ Python, nhóm mong muốn xây dựng một tài liệu tham khảo có

giá trị thực tiễn, kết hợp lý thuyết chặt chẽ với minh chứng bằng source code cụ thể và kết quả thực nghiệm trực quan.

1.2. Mục tiêu nghiên cứu

- Hệ thống hóa lý thuyết về đồ thị: Trình bày các khái niệm cơ bản (đỉnh, cạnh, trọng số, chu trình, bậc đỉnh), các loại đồ thị và cấu trúc dữ liệu biểu diễn đồ thị phổ biến.

- Phân tích chuyên sâu các thuật toán tiêu biểu: Nghiên cứu nguyên lý hoạt động, giả mã (pseudocode), độ phức tạp thời gian và không gian - bao gồm: nhóm duyệt đồ thị (BFS, DFS), nhóm đường đi ngắn nhất (Dijkstra, Bellman-Ford), nhóm cây khung nhỏ nhất (Kruskal, Prim), nhóm chu trình Euler (Fleury, Hierholzer) và luồng cực đại (Ford-Fulkerson).

- Cài đặt thuật toán bằng Python: Xây dựng source code rõ ràng, có chú thích đầy đủ, được tổ chức theo các module riêng biệt, có thể tái sử dụng và mở rộng.

- Thực nghiệm và đánh giá: Kiểm thử các thuật toán trên nhiều bộ dữ liệu JSON thực tế (đồ thị có hướng, vô hướng, trọng số âm, không liên thông...), ghi nhận kết quả và so sánh hiệu suất.

- Rút ra bài học ứng dụng: Xác định khi nào nên sử dụng thuật toán nào và đề xuất hướng cải tiến cho các nghiên cứu tiếp theo.

1.3. Phạm vi và giới hạn đề tài

1.3.1. Phạm vi nghiên cứu

Đề tài tập trung vào các nội dung sau:

- Lý thuyết đồ thị cơ bản: đồ thị có hướng, vô hướng, có trọng số, không có trọng số.

- Cấu trúc dữ liệu biểu diễn đồ thị: danh sách kề (Adjacency List) dưới dạng dict Python.

- Nhóm duyệt đồ thị: Breadth-First Search (BFS) và Depth-First Search (DFS).
- Nhóm đường đi ngắn nhất: Dijkstra (trọng số không âm) và Bellman-Ford (hỗ trợ trọng số âm, phát hiện chu trình âm).
- Nhóm cây khung nhỏ nhất (MST): Kruskal (dùng Union-Find) và Prim.
- Nhóm chu trình Euler: Fleury và Hierholzer.
- Luồng cực đại: Ford-Fulkerson.
- Cài đặt toàn bộ bằng Python 3.9+, sử dụng thư viện matplotlib và networkx để trực quan hóa.
- Kiểm thử trên các bộ dữ liệu JSON thực tế đa dạng (lưu trong thư mục test/).

1.3.2. Giới hạn đề tài

Để đảm bảo tính tập trung và chất lượng của báo cáo, đề tài có một số giới hạn nhất định:

- Thực nghiệm được thực hiện trên các đồ thị có kích thước vừa phải; chưa đánh giá hiệu suất ở quy mô lớn (hàng triệu đỉnh).
- Thuật toán Fleury có độ phức tạp $O(E^2)$ do kiểm tra cầu tại mỗi bước, chưa được tối ưu cho đồ thị dày đặc.

1.4. Phương pháp nghiên cứu

Nghiên cứu được thực hiện theo phương pháp kết hợp giữa lý thuyết và thực nghiệm, cụ thể qua bốn bước chính:

1.4.1. Nghiên cứu tài liệu

Nhóm tổng hợp kiến thức từ các giáo trình kinh điển như "Introduction to Algorithms" (CLRS — Cormen et al.), "Algorithm Design" (Kleinberg &

Tardos), và các tài liệu học thuật uy tín. Ngoài ra, tham khảo các tài nguyên trực tuyến từ GeeksforGeeks, Coursera và MIT OpenCourseWare.

1.4.2. Phân tích và mô tả thuật toán

- Mỗi thuật toán được trình bày theo cấu trúc thống nhất: ý tưởng cốt lõi → giả mã → phân tích độ phức tạp → ví dụ minh họa từng bước → ưu và nhược điểm.

1.4.3. Cài đặt và kiểm thử

Toàn bộ thuật toán được cài đặt bằng Python 3.9+, lưu trữ và quản lý phiên bản trên GitHub tại “github.com/HanaFS/graph_algorithms”. Source code được tổ chức thành các module riêng biệt theo từng nhóm thuật toán (traversal/, shortest_path/, minimum_spanning_tree/, advanced/), có chú thích rõ ràng. Kiểm thử được thực hiện trên nhiều bộ dữ liệu JSON đa dạng (đồ thị có hướng, vô hướng, trọng số âm, đồ thị Euler, mô phỏng mạng...) lưu trong thư mục test/.

1.5. Bố cục báo cáo

Báo cáo được tổ chức thành 5 chương chính:

- Chương 1 - Tổng quan: Lý do chọn đề tài, mục tiêu, phạm vi, phương pháp nghiên cứu và bố cục báo cáo.
- Chương 2 - Cơ sở lý thuyết: Hệ thống hóa lý thuyết đồ thị: định nghĩa, phân loại, cấu trúc biểu diễn và tổng quan các bài toán nền tảng.
- Chương 3 - Phân tích các thuật toán: Trình bày chi tiết từng thuật toán: nguyên lý, giả mã, độ phức tạp, ví dụ minh họa và so sánh.
- Chương 4 - Cài đặt và thực nghiệm: Môi trường cài đặt, kiến trúc module, kết quả thực nghiệm trên các bộ dữ liệu JSON và nhận xét đánh giá.
- Chương 5 - Kết luận: Tổng kết kết quả đạt được.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Lý thuyết đồ thị

2.1.1. Định nghĩa đồ thị

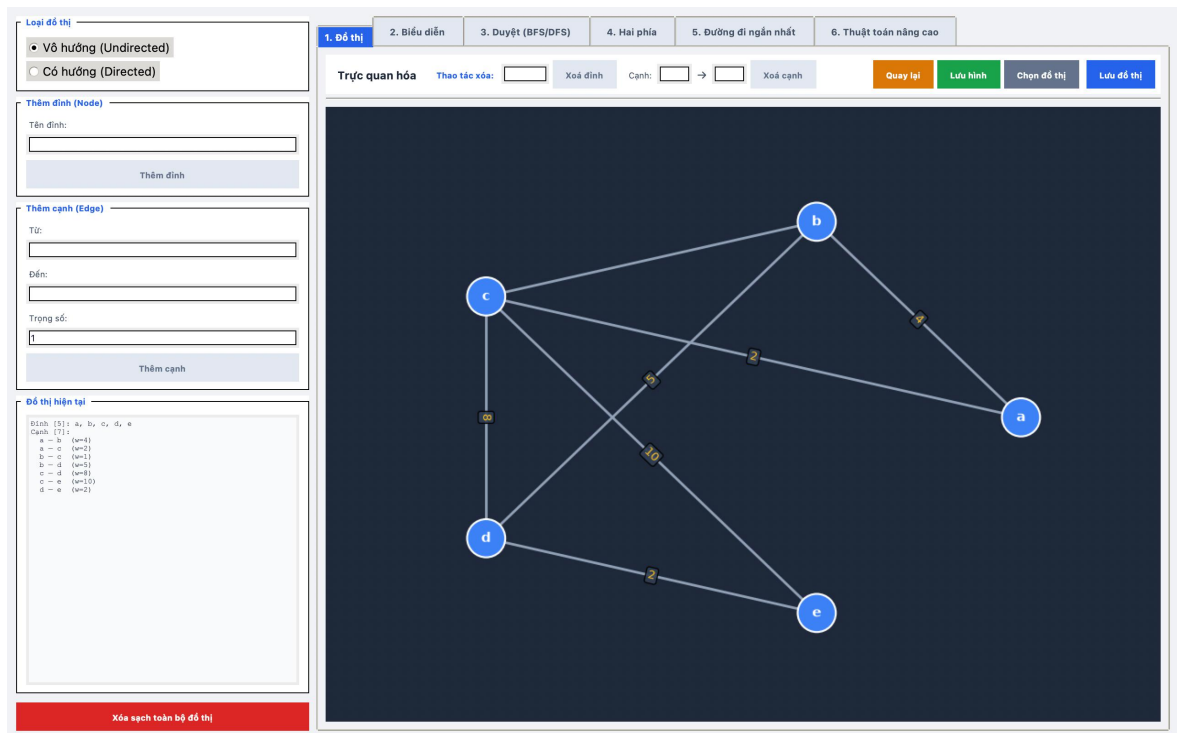
- Đồ thị G được định nghĩa bởi một cặp có thứ tự $G = (V, E)$:

+ V (Vertices / Nodes): Tập hợp hữu hạn, khác rỗng các đỉnh (nút).

Trong mã nguồn của dự án, mỗi đỉnh được định danh duy nhất bằng một chuỗi ký tự (string).

+ E (Edges / Arcs): Tập hợp các cạnh (cung). Mỗi cạnh e nối một cặp đỉnh (u, v) với $u, v \in V$. Trong đồ thị có hướng: $E \subseteq V \times V$, cạnh là cặp có thứ tự (u, v) . Trong đồ thị vô hướng: $(u, v) \equiv (v, u)$.

Mỗi cạnh có thể gắn kèm một giá trị trọng số (weight) kiểu số thực (float), mang ý nghĩa ngữ nghĩa linh hoạt tùy bài toán: khoảng cách địa lý, thời gian di chuyển, chi phí năng lượng hoặc sức chứa thông lượng mạng. Đồ thị trong dự án được khởi tạo qua class `Graph` (`core/graph.py`), với thuộc tính `directed: bool` kiểm soát loại đồ thị.



Hình 2.1. Giao diện trực quan hóa đồ thị

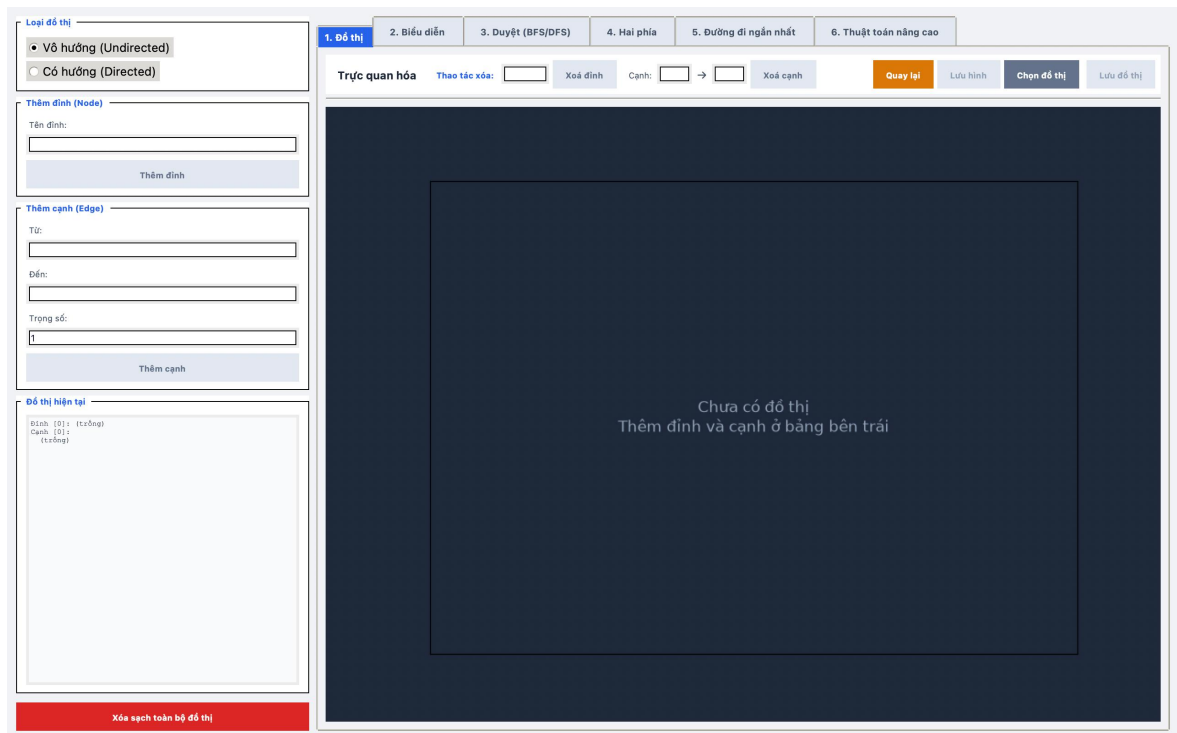
2.1.2. Phân loại đồ thị

Dự án hỗ trợ và làm việc với nhiều loại đồ thị khác nhau, được kiểm soát bởi thuộc tính `directed` trong class `Graph` và trong file dữ liệu JSON. Bảng sau tóm tắt các loại đồ thị được dùng trong codebase:

Loại đồ thị	Mô tả	Ví dụ trong dự án
Vô hướng (Undirected)	Cạnh $(u, v) = \text{cạnh}(v, u)$	test/1.json, UndirectedGraph-11.json
Có hướng (Directed)	Cạnh $(u, v) \neq \text{cạnh}(v, u)$	DirectedGraph-11.json, bài toán Max-Flow

Loại đồ thị	Mô tả	Ví dụ trong dự án
Có trọng số (Weighted)	Mỗi cạnh mang một giá trị số thực	Dijkstra, Bellman-Ford, Kruskal, Prim
Không có trọng số	Cạnh chỉ xác định kết nối	BFS, DFS (weight mặc định = 1)
Có trọng số âm	Tồn tại cạnh với $\text{weight} < 0$	(Directed)-Trọng số âm- có chutìnhâm.json
Không liên thông	Tồn tại đỉnh không thể đến được	(Undirected)-Không liên thông.json
Đồ thị Euler	Tồn tại đường/chu trình Euler	(Undirected)-Đồ thị Euler.json

Bảng 2.1. Phân loại các loại đồ thị sử dụng trong dự án



Hình 2.2. Giao diện khởi tạo đồ thị

2.1.3. Các khái niệm và định lý cơ bản

- Đỉnh kề (Neighbor): Đỉnh v được gọi là kề với u nếu tồn tại cạnh $(u, v) \in E$. Trong codebase, hàm `_neighbors()` trong `bfs.py` và `dfs.py` chịu trách nhiệm trích xuất danh sách đỉnh kề.

- Bậc đỉnh (Degree): Số lượng cạnh liên thuộc với đỉnh v , ký hiệu là $\deg(v)$. Theo Định lý bắt tay (Handshaking Lemma): $\sum_{v \in V} \deg(v) = 2 \times |E|$. Tổng số bậc của mọi đỉnh trong đồ thị vô hướng luôn là số chẵn. Trong đồ thị có hướng, phân biệt bán bậc vào $\deg^-(v)$ và bán bậc ra $\deg^+(v)$: $\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E|$.

- Đường đi (Path): Dãy đỉnh $P = (v_0, v_1, \dots, v_k)$ sao cho $(v_i, v_{i+1}) \in E$ với mọi i . Dijkstra và Bellman-Ford trả về dict previous để truy vết đường đi tối ưu từ đỉnh đích ngược về đỉnh nguồn.

- Chu trình (Cycle): Đường đi khép kín có đỉnh đầu trùng đỉnh cuối ($v_0 = v_k, k \geq 3$). Kruskal dùng cấu trúc Disjoint Set Union (Union-Find) để phát hiện và phòng tránh tạo chu trình khi xây dựng cây khung nhỏ nhất.

- Chu trình âm (Negative Cycle): Chu trình C có tổng trọng số: $\sum_{(e \in C)} w(e) < 0$. Khi xuất hiện chu trình âm, bài toán đường đi ngắn nhất không xác định được cực tiểu hữu hạn. Module bellman_ford.py phát hiện và chủ động ném ngoại lệ: `ValueError("Đồ thị chứa chu trình âm có thể đi tới được từ đỉnh xuất phát!")`.

- Đồ thị liên thông: Mọi cặp đỉnh đều có đường đi nối với nhau (áp dụng cho đồ thị vô hướng). BFS và DFS được dùng để kiểm tra tính liên thông trong nhiều module.

- Cầu (Bridge / Cut-edge): Cạnh e mà khi loại bỏ khỏi đồ thị, số thành phần liên thông tăng lên. Đây là khái niệm cốt lõi trong thuật toán Fleury, được cài đặt qua phương thức `_is_bridge(u, v)` bằng cách chạy kiểm tra liên thông bằng DFS trước và sau khi tạm gỡ cạnh.

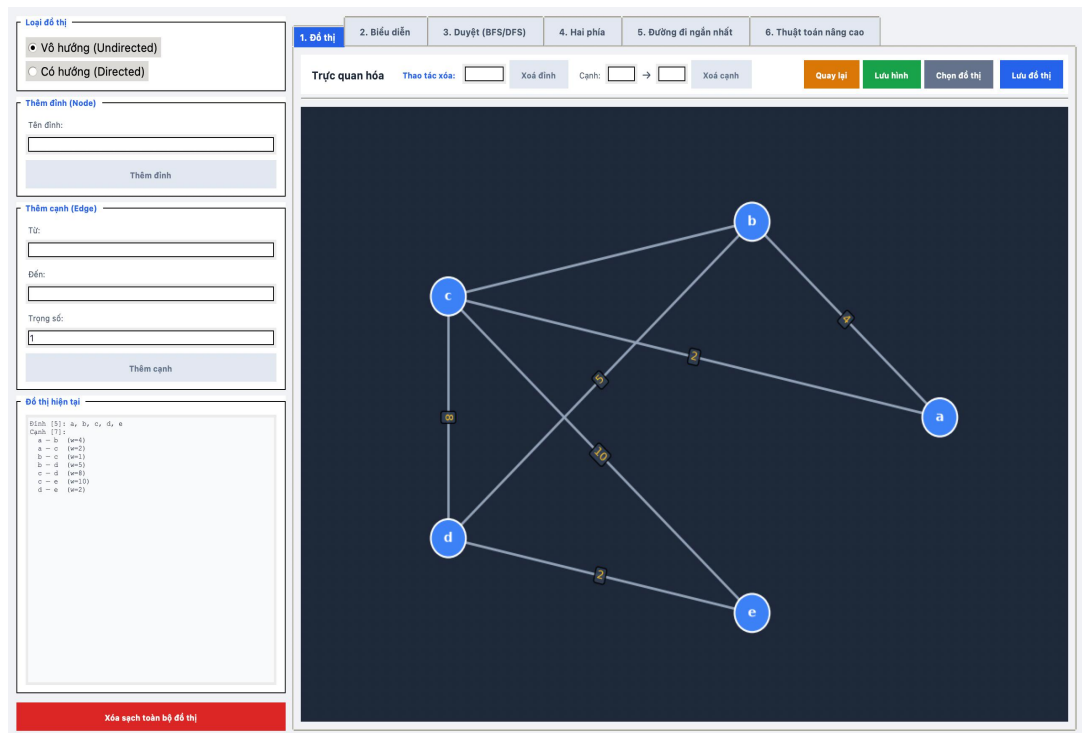
2.2. Cấu trúc dữ liệu biểu diễn đồ thị

2.2.1. Danh sách kề (Adjacency List)

Trong toàn bộ dự án, đồ thị được biểu diễn thống nhất bằng Danh sách kề (Adjacency List) - hiện thực hóa qua từ điển băm (dict) của Python. Mỗi đỉnh là một khóa kiểu chuỗi trỏ tới danh sách các bộ giá trị (đỉnh_kề, trọng_số).

Ưu thế kỹ thuật so với Ma trận kề:

- Tiết kiệm bộ nhớ: Chi phí không gian $O(|V| + |E|)$, triệt tiêu lãng phí bộ nhớ trên đồ thị thưa ($|E| \ll |V|^2$) so với ma trận kề tốn $O(|V|^2)$.
- Duyệt láng giềng nhanh: Quét qua tập đỉnh kề của u chỉ tốn $O(\deg(u))$
- không cần duyệt toàn bộ $|V|$ đỉnh.
- Tra cứu tức thời: Bảng băm tích hợp sẵn trong dict Python giúp truy xuất danh sách kề đạt thời gian trung bình $O(1)$.
- Tương thích toàn codebase: Cả 9 thuật toán đều nhận cấu trúc này làm đầu vào, đảm bảo tính nhất quán cao.



Hình 2.3. Giao diện hiển thị dữ liệu đồ thị

2.2.2. Định dạng file dữ liệu đầu vào (JSON)

Dữ liệu kiểm thử được tổ chức độc lập dưới định dạng chuẩn JSON - tách biệt dữ liệu và logic xử lý (Separation of Concerns). Điều này cho phép kiểm thử linh hoạt mà không cần sửa đổi mã nguồn:

```
{
  "directed": false,
  "graph": {
    "a": [["b", 4], ["c", 2]],
    "b": [["a", 4], ["c", 1], ["d", 5]],
    "c": [["a", 2], ["b", 1], ["d", 8], ["e", 10]],
    "d": [["b", 5], ["c", 8], ["e", 2]],
    "e": [["c", 10], ["d", 2]]
  }
}
```

- Trường "directed" được đọc để cập nhật trạng thái giao diện (biến self.directed trong tab_graph.py). Dữ liệu cạnh được nạp từ JSON - file đồ thị vô hướng đã lưu sẵn cạnh hai chiều (ví dụ: test/1.json lưu đủ cả ["b", 4] trong danh sách của "a" và ["a", 4] trong danh sách của "b").

- Trường "graph": Chứa danh sách kề theo định dạng [[neighbor, weight]] với giá trị trọng số nguyên hoặc thực. Các file kiểm thử trong thư mục test/ bao phủ đầy đủ các kịch bản đồ thị thực tế.

2.2.3. Đồ thị thặng dư (Residual Graph) trong Ford-Fulkerson

Riêng thuật toán Ford-Fulkerson (Edmonds-Karp) sử dụng thêm cấu trúc Đồ thị thặng dư (Residual Graph) - khác hoàn toàn với Adjacency List thông thường. Cấu trúc này được biểu diễn bằng từ điển lồng nhau hai cấp.

Cung nghịch (backward edge) cho phép triệt tiêu hoặc chuyển hướng lượng luồng đã gửi trước đó - đây là yếu tố cốt lõi đảm bảo thuật toán đạt được luồng cực đại toàn cục.

2.3. Tổng quan các nhóm bài toán và thuật toán

2.3.1. Nhóm duyệt đồ thị (traversal/)

Bài toán duyệt yêu cầu ghé thăm tất cả các đỉnh có thể tiếp cận được từ đỉnh xuất phát theo trật tự xác định, đảm bảo mỗi nút chỉ được xử lý đúng một lần.

Thuật toán	File	Độ phức tạp
BFS (Breadth-First Search)	bfs.py	$O(V + E)$
DFS (Depth-First Search)	dfs.py	$O(V + E)$

Bảng 2.2. So sánh BFS và DFS

Hàm tiện ích `_neighbors()` trong cả hai file tự động trích xuất danh sách đỉnh kề từ cấu trúc `[(neighbor, weight)]` hoặc `[neighbor]`. DFS được cài đặt dạng lặp (iterative) sử dụng ngăn xếp Python để phòng ngừa triệt để lỗi tràn bộ nhớ (Call Stack Overflow).

Hình 2.5. Giao diện thực hiện thuật toán BFS và DFS

2.3.2. Nhóm đường đi ngắn nhất (*shortest_path/*)

Bài toán đường đi ngắn nhất nguồn đơn (Single-Source Shortest Path): Cho đồ thị có trọng số $G = (V, E, w)$ và đỉnh nguồn s , tìm đường đi có tổng trọng số nhỏ nhất từ s tới các đỉnh còn lại.

Thuật toán	File	Điều kiện áp dụng & Cấu trúc	Độ phức tạp
Dijkstra	dijkstra.py	Trọng số không âm ($w \geq 0$), ném ValueError nếu có cạnh âm. Dùng Min-Heap (heapq).	$O((V + E) \log V)$
Bellman-Ford	bellman_ford.py	Hỗ trợ trọng số âm, phát hiện chu trình âm qua $ V -1$ lần lặp relaxation.	$O(V \times E)$

Bảng Tổng quan thuật toán Dijkstra và Bellman-Ford

Cơ chế nới lỏng cạnh (Relaxation): Nếu $\text{dist}[u] + w(u, v) < \text{dist}[v]$ thì cập nhật $\text{dist}[v] = \text{dist}[u] + w(u, v)$ và $\text{previous}[v] = u$. Cả hai module đều trả về cặp (distances, previous) để truy vết đường đi hoàn chỉnh.

The screenshot shows a web application for finding shortest paths. On the left, there's a sidebar with sections: 'Loại đồ thị' (Undirected/Directed), 'Thêm đỉnh (Node)' (Add Node), 'Thêm cạnh (Edge)' (Add Edge), and 'Đồ thị hiện tại' (Current Graph). The main area has tabs for '1. Đồ thị', '2. Biểu diễn', '3. Duyệt (BFS/DFS)', '4. Hai phía', '5. Đường đi ngắn nhất' (selected), and '6. Thuật toán nâng cao'. Under the 'Shortest Path' tab, there's a title 'Tìm đường đi ngắn nhất (Dijkstra & Bellman-Ford)'. Below this are input fields for 'Đỉnh nguồn (Source): a' and 'Đỉnh đích (Target): e', with buttons for 'Dijkstra' and 'Bellman-Ford'. Two result panels are shown: 'Dijkstra' and 'Bellman-Ford'. Both display the same result: 'Kết quả thuật toán: Đường đi: a -> c -> b -> d -> e, Tổng chi phí: 10'. At the bottom of the sidebar, there's a red button 'Xóa sạch toàn bộ đồ thị'.

Hình 2.6. Giao diện thuật toán Dijkstra và Bellman-Ford

2.3.3. Nhóm cây khung nhỏ nhất (*minimum spanning tree*)

Bài toán Cây khung nhỏ nhất (Minimum Spanning Tree — MST): Tìm tập con các cạnh không chứa chu trình kết nối toàn bộ $|V|$ đỉnh của đồ thị vô hướng sao cho tổng trọng số các cạnh là nhỏ nhất: $\min \sum_{(e \in T)} w(e)$

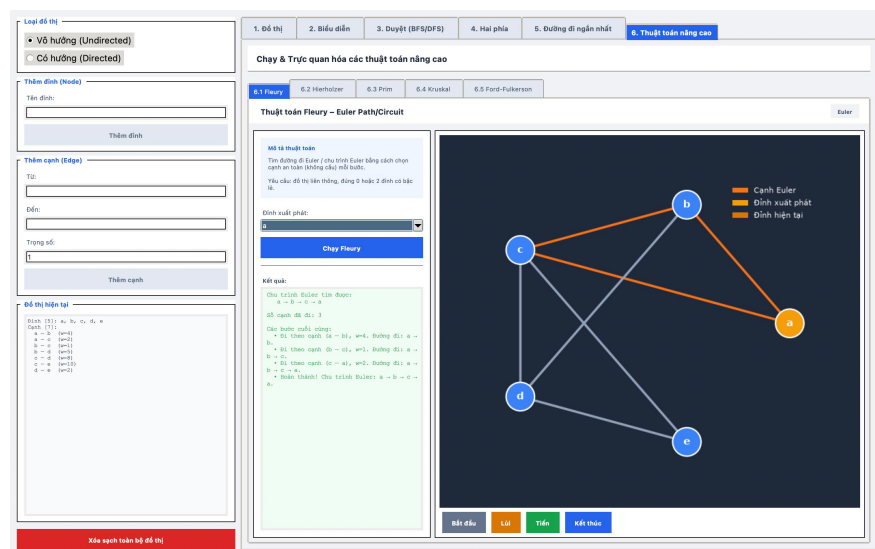
Thuật toán	File	Kỹ thuật & Cấu trúc dữ liệu	Độ phức tạp
Kruskal	kruskal.py	Tiếp cận theo cạnh. Dùng Union-Find với Path Compression và Union by Rank.	$O(E \log E)$
Prim	prim.py	Tiếp cận theo đỉnh. Dùng	$O(E \log V)$

Thuật toán	File	Kỹ thuật & Cấu trúc dữ liệu	Độ phức tạp
		Min-Heap (heapq) với itertools.count làm tie- breaker.	

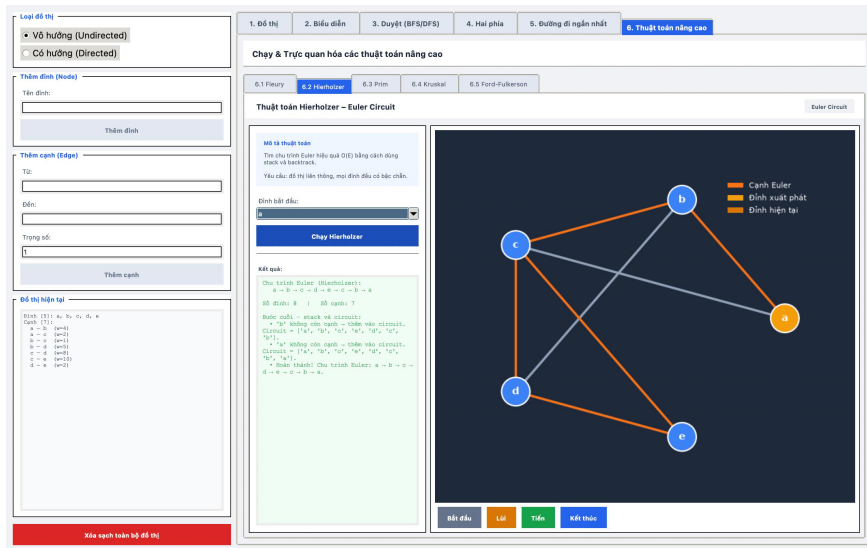
Bảng Tổng quan thuật toán Kruskal và Prim

Kruskal đạt tốc độ gần như $O(1)$ cho mỗi thao tác kiểm tra chu trình nhờ cấu trúc tập hợp rời rạc tối ưu. Prim trong dự án xử lý an toàn cả trường hợp đồ thị có cạnh mang trọng số âm. Cả hai đều ghi nhận mảng steps hỗ trợ diễn hoạt hoạt họa trên giao diện.

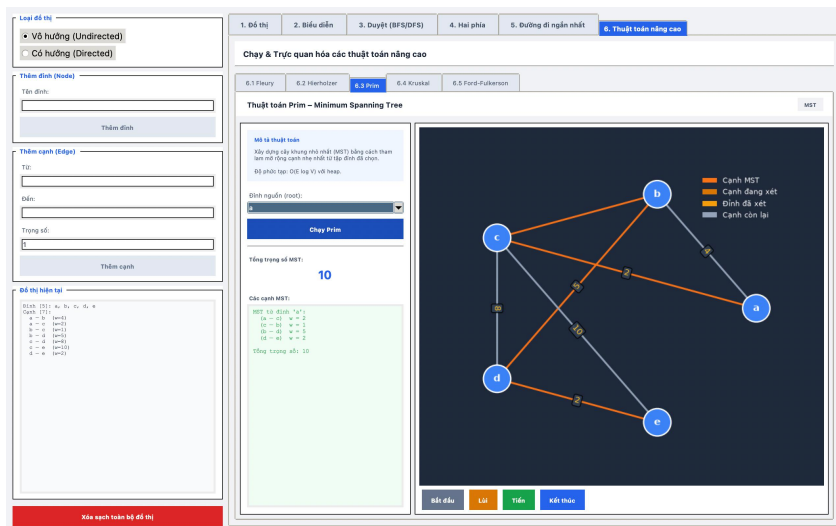
2.3.4. Nhóm thuật toán nâng cao (advanced/)



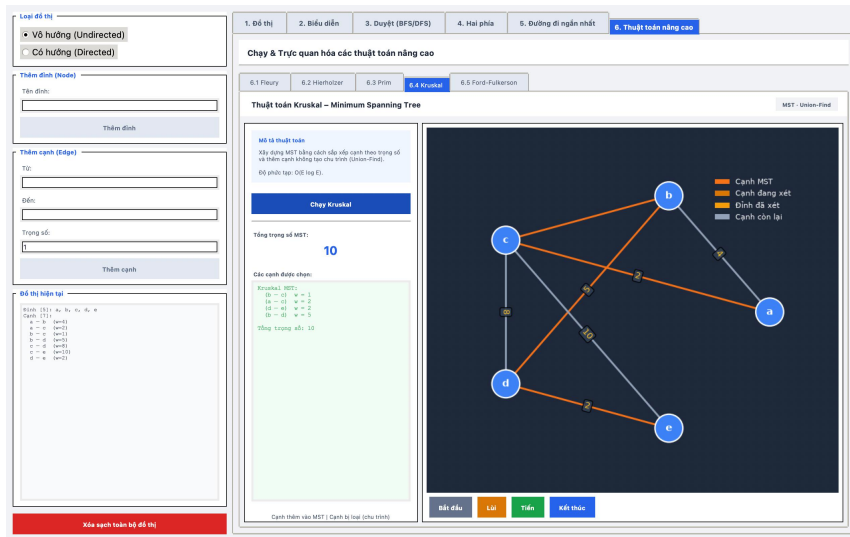
Hình 2.8. Giao diện trực quan hóa thuật toán nâng cao



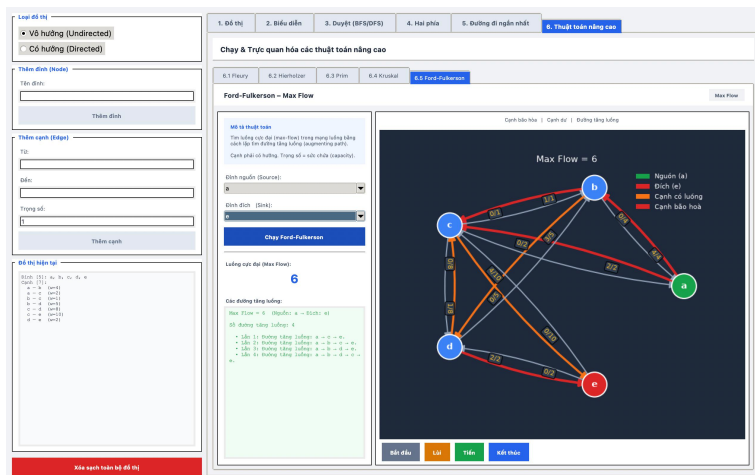
Hình 2.9. Giao diện thực hiện thuật toán Fleury



Hình 2.10. Giao diện thực hiện thuật toán Hierholzer



Hình 2.11. Giao diện thực hiện thuật toán Ford-Fulkerson



2.4. Kiến trúc mã nguồn tổng thể của dự án

Dự án được tổ chức phân lớp rõ ràng, tách bạch giữa tầng dữ liệu cốt lõi, tầng thuật toán thuần và tầng giao diện đồ họa người dùng Tkinter:

```
graph_algorithms/
├── main.py
├── pyproject.toml
├── src/graph_algorithms/
│   └── core/
```

```
| | └─ graph.py
| └─ algorithms/
|   └─ traversal/
|   └─ shortest_path/
|   └─ minimum_spanning_tree/
|   └─ advanced/
└─ ui/
    └─ graph_ui.py
    └─ sidebar.py
    └─ theme.py
    └─ tab_graph.py
    └─ tab_repr.py
    └─ tab_traversal.py
    └─ tab_bipartite.py
    └─ tab_shortest.py
    └─ tab_advanced.py
└─ test/
```

CHƯƠNG 3: PHÂN TÍCH CÁC THUẬT TOÁN

3.1. Thuật toán BFS (Breadth-First Search)

3.1.1. Ý tưởng

BFS duyệt đồ thị theo chiều rộng — thăm tất cả đỉnh ở cùng một "lớp" (khoảng cách từ gốc) trước khi đi sâu hơn. Dùng hàng đợi (FIFO) để quản lý thứ tự thăm đỉnh.

3.1.2. Code

```
visited = {start}
queue = deque([start])
order: List[Any] = []

while queue:
    u = queue.popleft()
    order.append(u)
    for neighbor in _neighbors(adj, u):
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)

return order
```

Hình 3.1. Mã nguồn thuật toán BFS

3.1.3. Phân tích độ phức tạp

Độ phức tạp	Giá trị	Lý do
Thời gian	$O(V + E)$	Mỗi đỉnh thêm vào hàng đợi và xử lý đúng 1 lần; mỗi cạnh được xét đúng 1 lần

Độ phức tạp	Giá trị	Lý do
Không gian	$O(V)$	Hàng đợi chứa tối đa $ V $ đỉnh; tập visited kích thước $ V $

Bảng 3.1. Độ phức tạp của thuật toán BFS

3.1.4. Ưu và nhược điểm

Ưu điểm	Nhược điểm
Tìm đường đi ngắn nhất (theo số cạnh) trên đồ thị không có trọng số	Tốn bộ nhớ hơn DFS khi đồ thị rộng (nhiều đỉnh cùng lớp)
Đảm bảo thăm đỉnh gần trước — phù hợp kiểm tra liên thông	Không tối ưu cho đồ thị sâu với nhánh dài
Kết quả ổn định nhờ sắp xếp láng giềng	Không phù hợp để phát hiện chu trình trực tiếp

Bảng 3.2. Ưu và nhược điểm của thuật toán BFS

3.2. Thuật toán DFS (Depth-First Search)

3.2.1. Ý tưởng

DFS duyệt theo chiều sâu — đi sâu nhất có thể theo một nhánh trước khi quay lui. Trong dự án, DFS được cài đặt theo kiểu lặp (iterative) dùng stack thay vì đệ quy, tránh lỗi tràn ngăn xếp gọi hàm (Call Stack Overflow) trên đồ thị lớn.

3.2.2. Giải mã

```

visited: set = set()
stack = [start]
order: List[Any] = []

while stack:
    u = stack.pop()
    if u in visited:
        continue
    visited.add(u)
    order.append(u)

    # Đẩy hàng xóm theo thứ tự ngược để thứ tự pop đúng
    for neighbor in reversed(_neighbors(adj, u)):
        if neighbor not in visited:
            stack.append(neighbor)

return order

```

Hình 3.2. Mã nguồn thuật toán DFS

3.2.3. Phân tích độ phức tạp

Độ phức tạp	Giá trị	Lý do
Thời gian	$O(V + E)$	Mỗi đỉnh được xét đúng 1 lần; mỗi cạnh được xét đúng 1 lần
Không gian	$O(V)$	Stack chứa tối đa $ V $ đỉnh trong trường hợp đồ thị là đường thẳng

Bảng 3.4. Độ phức tạp của thuật toán DFS

3.2.4. Ưu và nhược điểm

Ưu điểm	Nhược điểm
Tiết kiệm bộ nhớ hơn BFS trên đồ thị sâu	Không tìm được đường đi ngắn nhất
Phù hợp phát hiện chu trình, kiểm tra cầu (dùng trong Fleury)	Kết quả phụ thuộc vào thứ tự duyệt
Cài đặt iterative tránh tràn ngăn xếp	Có thể đi sâu vô hạn nếu không kiểm soát visited

Bảng 3.5. Ưu và nhược điểm của thuật toán DFS

3.3. Thuật toán Dijkstra

3.3.1. Ý tưởng

Dijkstra tìm đường đi ngắn nhất từ một đỉnh nguồn đến tất cả các đỉnh còn lại, áp dụng cho đồ thị không có cạnh trọng số âm. Dùng hàng đợi ưu tiên (min-heap) để luôn xử lý đỉnh có khoảng cách tạm thời nhỏ nhất trước.

3.3.2. Code

```

distances = {node: float('inf') for node in graph}
distances[start] = 0
previous = {node: None for node in graph}

# Priority queue lưu (distance, node)
pq = [(0, start)]

while pq:
    current_distance, current_node = heapq.heappop(pq)

    if current_distance > distances[current_node]:
        continue

    if target and current_node == target:
        break

    for neighbor, weight in graph.get(current_node, []):
        if weight < 0:
            raise ValueError("Dijkstra không hoạt động với đồ thị có trọng số âm!")

        distance = current_distance + weight

        if distance < distances.get(neighbor, float('inf')):
            distances[neighbor] = distance
            previous[neighbor] = current_node
            heapq.heappush(pq, item: (distance, neighbor))

return distances, previous

```

Hình 3.3. Mã nguồn thuật toán Dijkstra

3.3.3. Phân tích độ phức tạp

Độ phức tạp	Giá trị	Ghi chú
Thời gian	$O((V + E) \log V)$	Mỗi đỉnh heappop $O(\log V)$; mỗi cạnh relaxation + heappush $O(\log V)$
Không gian	$O(V + E)$	heap chứa tối đa $O(E)$ phần tử; bảng distances và previous $O(V)$

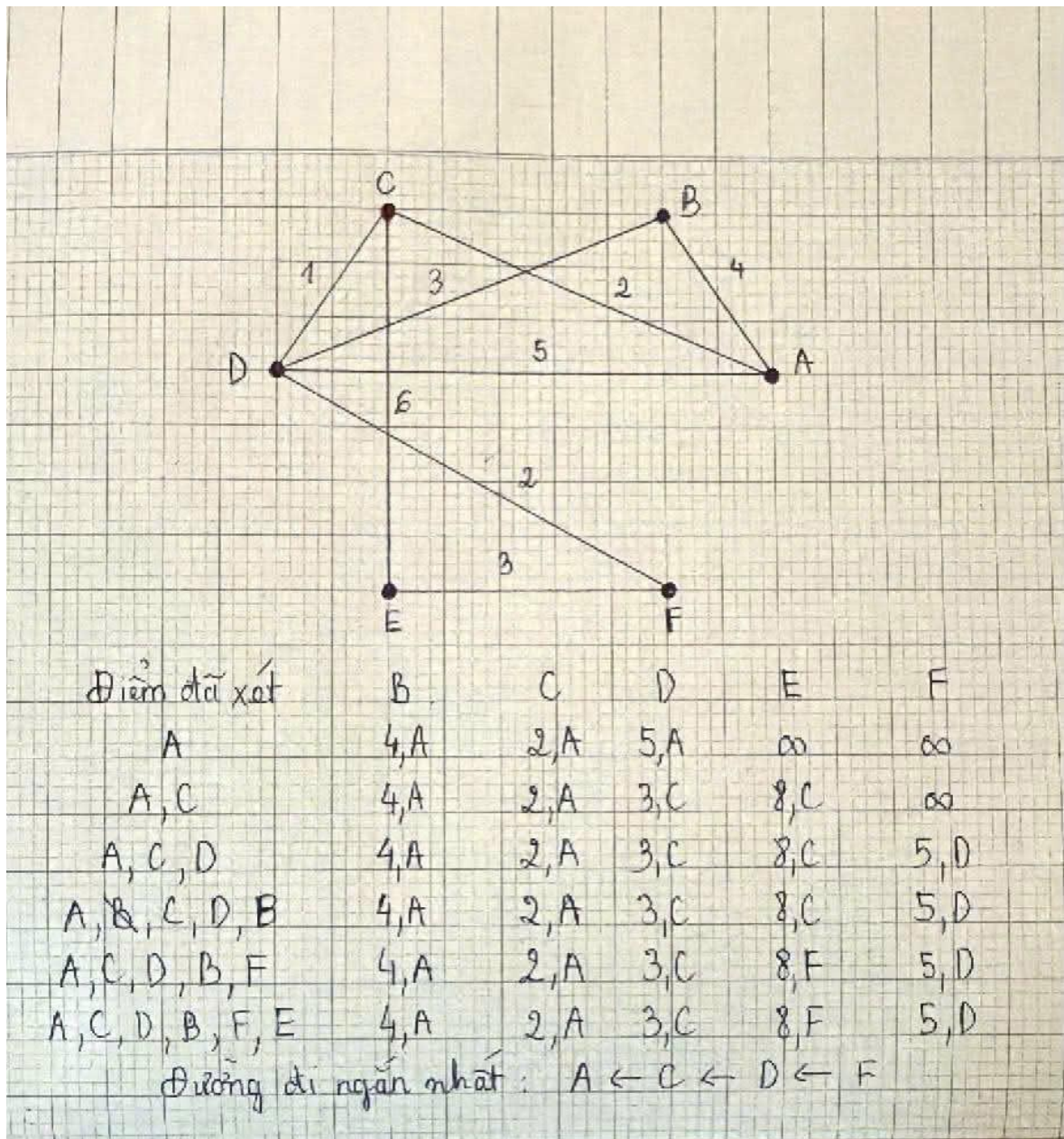
Bảng Độ phức tạp của thuật toán Dijkstra

3.3.4. Ưu và nhược điểm

Ưu điểm	Nhược điểm
Hiệu quả cao với heap nhị phân: $O((V+E)\log V)$	Không hoạt động với cạnh trọng số âm
Dừng sớm khi tìm đích (tham số target)	Không phát hiện chu trình âm
Tái sử dụng previous để truy vết đường đi	Kém hiệu quả khi đồ thị dày đặc so với Dijkstra dùng Fibonacci heap

Bảng Ưu và nhược điểm của thuật toán Dijkstra

3.3.5 Giải tay



3.4. Thuật toán Bellman-Ford

3.4.1. Ý tưởng

Bellman-Ford giải bài toán đường đi ngắn nhất với trọng số âm bằng cách lặp $|V|-1$ lần relaxation toàn bộ cạnh. Lần lặp thứ $|V|$ thêm để phát hiện chu trình âm.

3.4.2. Code

```

distances = {node: float('inf') for node in graph}
distances[start] = 0
previous = {node: None for node in graph}

nodes = list(graph.keys())

# V - 1 lần lặp (V là số đỉnh)
for _ in range(len(nodes) - 1):
    for u in nodes:
        if distances[u] == float('inf'):
            continue
        for v, weight in graph.get(u, []):
            if distances[u] + weight < distances.get(v, float('inf')):
                distances[v] = distances[u] + weight
                previous[v] = u

# Lần lặp thứ V để kiểm tra chu trình âm
for u in nodes:
    if distances[u] == float('inf'):
        continue
    for v, weight in graph.get(u, []):
        if distances[u] + weight < distances.get(v, float('inf')):
            # Phát hiện chu trình âm
            raise ValueError("Đồ thị chứa chu trình âm có thể đi tới được từ đỉnh xuất phát!")

return distances, previous

```

Hình 3.4. Mã nguồn thuật toán Bellman-Ford

3.4.3. Phân tích độ phức tạp

Độ phức tạp	Giá trị	Lý do
Thời gian	$O(V \times E)$	$ V -1$ vòng lặp, mỗi vòng xét $ E $ cạnh
Không gian	$O(V)$	Bảng distances và previous kích thước $ V $

Bảng Độ phức tạp của thuật toán Bellman-Ford

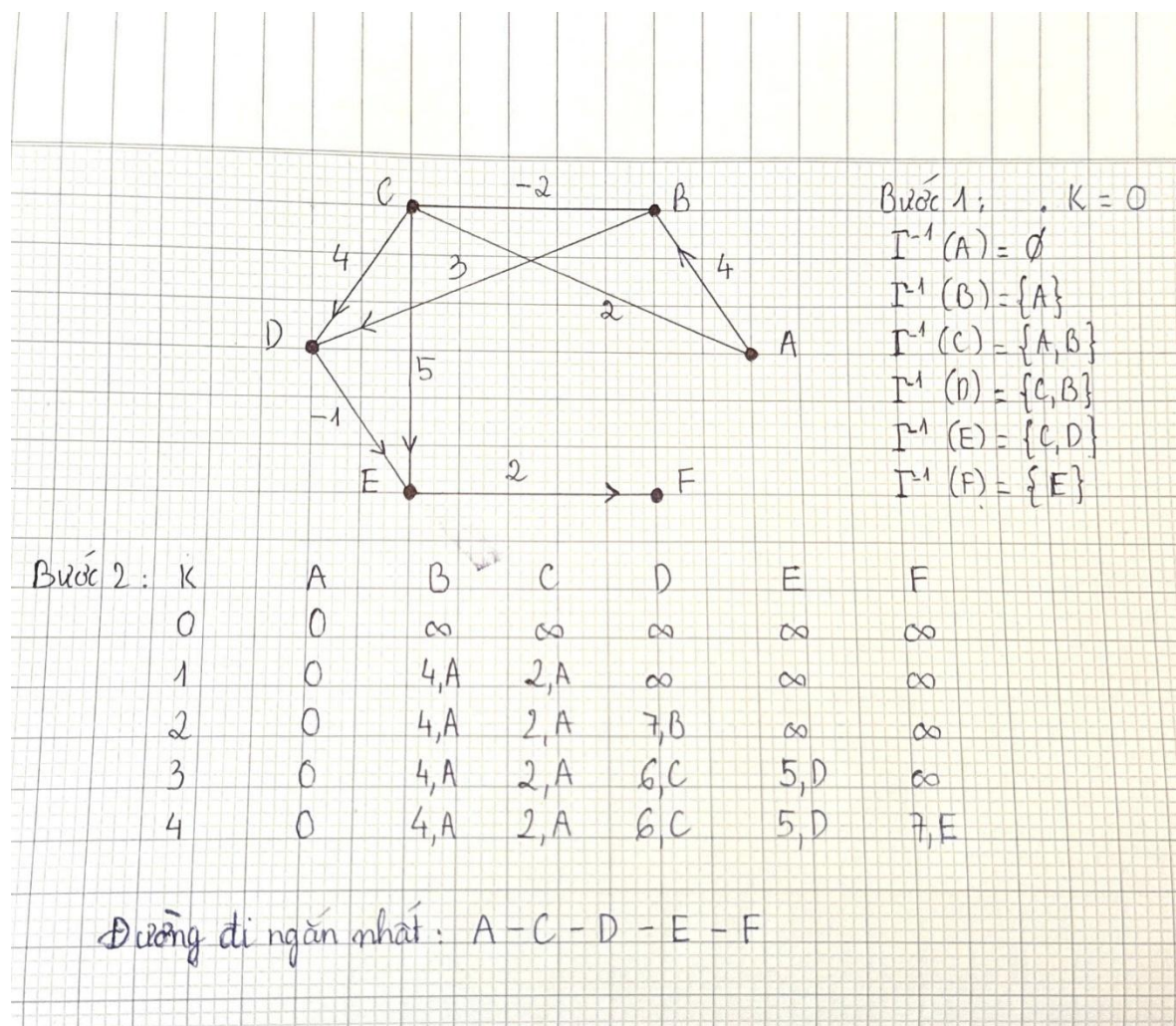
3.4.4. So sánh Dijkstra và Bellman-Ford

Tiêu chí	Dijkstra	Bellman-Ford
Trọng số âm	Không hỗ trợ	Hỗ trợ

Tiêu chí	Dijkstra	Bellman-Ford
Phát hiện chu trình âm	Không	Có
Độ phức tạp	$O((V+E)\log V)$	$O(V \times E)$
Phù hợp khi	Đồ thị lớn, trọng số không âm	Đồ thị có trọng số âm hoặc cần kiểm tra chu trình âm

Bảng So sánh thuật toán Dijkstra và Bellman-Ford

3.4.5 Giải tay



3.5. Thuật toán Kruskal

3.5.1. Ý tưởng

Kruskal xây dựng MST bằng chiến lược tham lam trên tập cạnh (Edge-centric): sắp xếp toàn bộ cạnh theo trọng số tăng dần, lần lượt thêm cạnh nhẹ nhất nếu không tạo chu trình. Sử dụng cấu trúc Union-Find (DSU) để kiểm tra chu trình trong $O(\alpha(V)) \approx O(1)$.

3.5.2. Code

```
# — Union-Find —
class UnionFind: 2 usages  GiaHan-coder
    def __init__(self, nodes):  GiaHan-coder
        self.parent = {n: n for n in nodes}
        self.rank = {n: 0 for n in nodes}

    def find(self, x: str) -> str: 3 usages  GiaHan-coder
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path compression
        return self.parent[x]

    def union(self, x: str, y: str) -> bool: 1 usage  GiaHan-coder
        """Hợp nhất 2 tập. Trả về True nếu hợp nhất thành công (không tạo chu trình)."""
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False # đã cùng tập → tạo chu trình
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        return True
```

Hình 3.5. Mã nguồn thuật toán Kruskal

3.5.3. Phân tích độ phức tạp

Độ phức tạp	Giá trị	Lý do
Thời	$O(E \log)$	Sắp xếp $ E $ cạnh chiếm $O(E \log E)$; tổng $O(E \cdot \alpha(V))$

Độ phức tạp	Giá trị	Lý do
gian	$ E)$	cho Union-Find $\approx O(E)$
Không gian	$O(V + E)$	Lưu danh sách cạnh $O(E)$, Union-Find $O(V)$

Bảng Độ phức tạp của thuật toán Kruskal

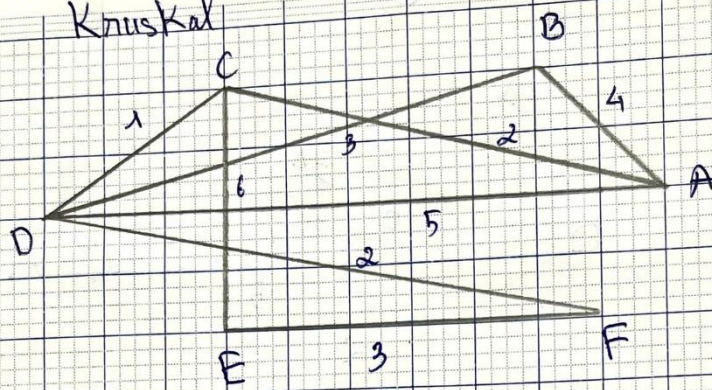
3.5.4. Ưu và nhược điểm

Ưu điểm	Nhược điểm
Hiệu quả cho đồ thị thưa (ít cạnh)	Cần sắp xếp toàn bộ cạnh trước
Dễ cài đặt và hiểu	Kém hiệu quả hơn Prim trên đồ thị dày đặc
Union-Find với path compression gần $O(1)$ mỗi thao tác	Cần xử lý đặc biệt cho đồ thị không liên thông

Bảng Ưu và nhược điểm của thuật toán Kruskal

3.5.5. Giải tay

Kruskal



1

$D \rightarrow C$ x

2

$C \rightarrow A$ x

2

$D \rightarrow F$ +

3

$D \rightarrow B$ +

3

$E \rightarrow F$ x

4

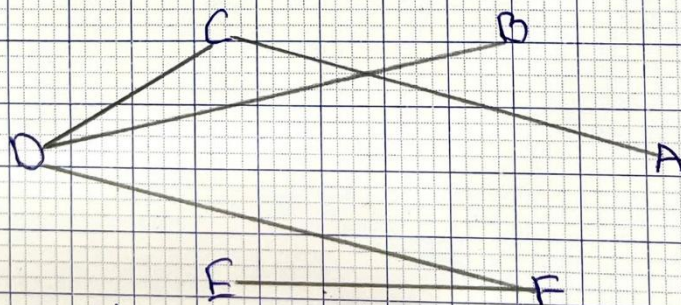
$B \rightarrow A$

5

$D \rightarrow A$

6

$C \rightarrow E$



$\Rightarrow$ CKNN $1+2+2+3+3=11$

HL BÚT CẦU

3.6. Thuật toán Prim

3.6.1. Ý tưởng

Prim xây dựng MST bằng chiến lược tham lam trên tập đỉnh (Vertex-centric): bắt đầu từ một đỉnh gốc, lặp lại việc thêm cạnh nhẹ nhất nối từ tập đỉnh đã chọn ra ngoài. Dùng min-heap để luôn chọn cạnh nhỏ nhất hiệu quả.

3.6.2. Code

```

while heap and len(visited) < len(graph):
    w, _, u, v = heapq.heappop(heap)    # bỏ tie-breaker _

    if v in visited:
        continue    # bỏ qua cạnh "stale" (v đã vào MST qua đường khác)

    # Thêm cạnh (u → v) vào MST
    visited.add(v)
    mst_edges.append((u, v, w))
    total_cost += w
    _push_edges(v)

    steps.append({
        "added_edge": (u, v, w),
        "visited": set(visited),
        "mst_edges": list [tuple[str, str, float | int]] (mst_edges),
        "description": f"Thêm cạnh ({u} - {v}), trọng số = {w}.",
    })

# Kiểm tra đồ thị có liên thông không
if len(visited) < len(graph):
    missing = set [str] (graph) - visited
    steps.append({
        "added_edge": None,
        "visited": set(visited),
        "mst_edges": list [tuple[str, str, float | int]] (mst_edges),
        "description": (
            f"Đồ thị không liên thông. "
            f"Không thể tới: {'', '}.join(sorted(missing))}."
        ),
    })

return mst_edges, total_cost, steps

```

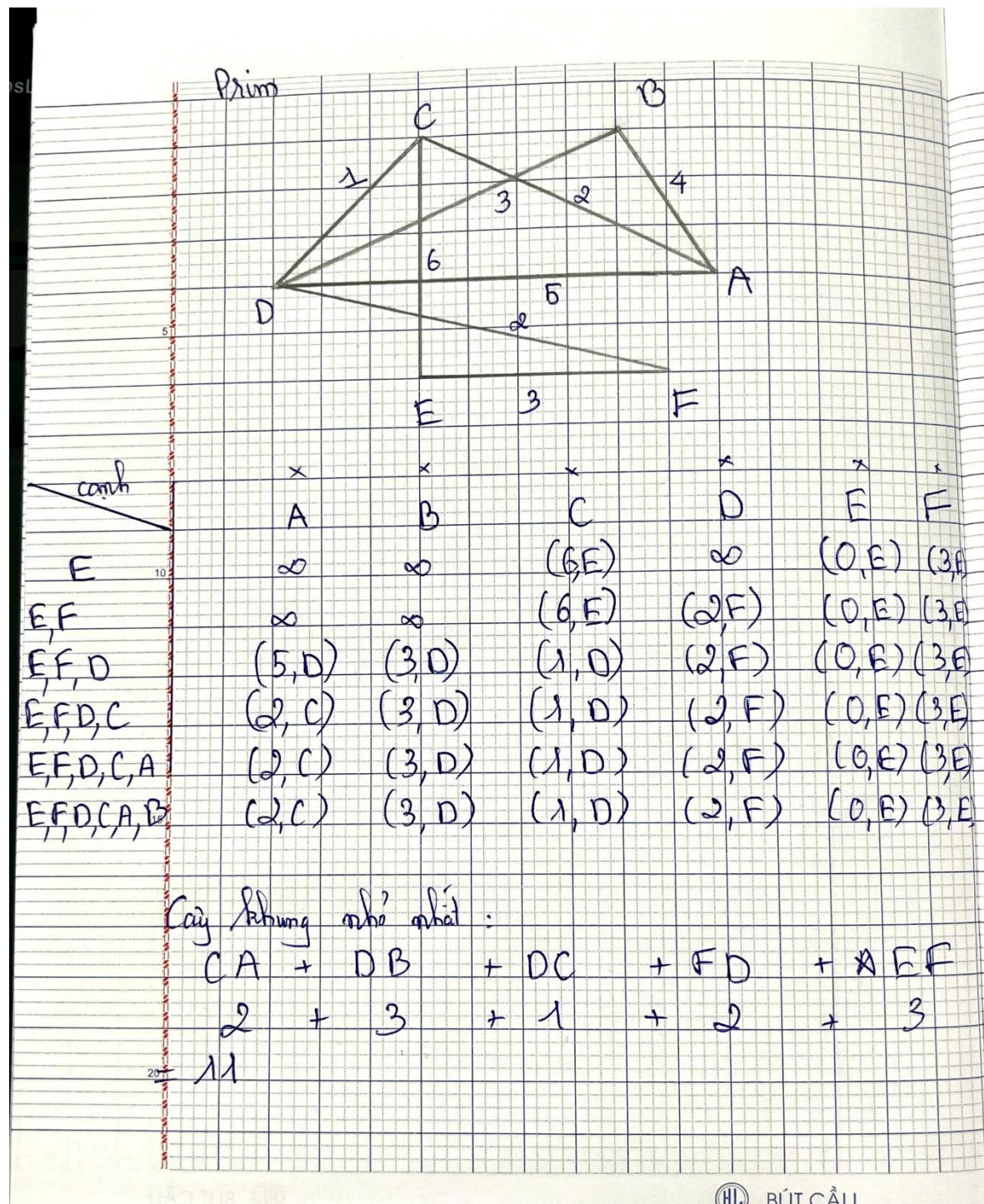
Hình 3.6. Mã nguồn thuật toán Prim

3.6.3. Phân tích độ phức tạp

Độ phức tạp	Giá trị	Lý do
Thời gian	$O(E \log V)$	Mỗi cạnh được heappush $O(\log V)$; tổng $ E $ lần push
Không gian	$O(V + E)$	heap $O(E)$, visited $O(V)$

Bảng Độ phức tạp của thuật toán Prim

3.6.3. Giải tay



3.6.4. So sánh Kruskal và Prim

Tiêu chí	Kruskal	Prim
Chiến lược	Edge-centric (xét tất cả cạnh)	Vertex-centric (mở rộng từ gốc)
Phù hợp	Đồ thị thưa ($ E $ nhỏ)	Đồ thị dày đặc ($ E $ lớn)
Trọng số âm	Hoạt động	Hoạt động (khác Dijkstra)
Kết quả đồ thị không liên thông	Trả về MST của từng thành phần	Trả về MST thành phần chứa root, báo đỉnh không tới được

Bảng So sánh thuật toán Kruskal và Prim

3.7. Thuật toán Fleury

3.7.1. Ý tưởng

Fleury tìm đường đi/chu trình Euler bằng cách thận trọng: tại mỗi bước, ưu tiên chọn cạnh không phải cầu (bridge), chỉ đi qua cầu khi không còn lựa chọn khác. Điều này đảm bảo không "chia cắt" đồ thị sớm.

3.7.2. Điều kiện tiên quyết

Trước khi chạy Fleury, hàm `check_euler_condition()` trong `fleury.py` kiểm tra:

- Chu trình Euler: Mọi đỉnh có bậc chẵn $\rightarrow$ xuất phát từ bất kỳ đỉnh nào.
- Đường đi Euler: Đúng 2 đỉnh bậc lẻ $\rightarrow$ xuất phát từ 1 trong 2 đỉnh đó.
- Ném `ValueError` nếu không thỏa điều kiện.

3.7.3. Code

```

# Xoá cạnh (current → chosen_nb) và ngược lại
g[current] = [(nb, w) for nb, w in g[current] if nb != chosen_nb]
g[chosen_nb] = [(nb, w) for nb, w in g[chosen_nb] if nb != current]

path.append(chosen_nb)
steps.append({
    "current": chosen_nb,
    "edge": (current, chosen_nb, chosen_w),
    "path": list[str](path),
    "description": (
        f"Đi theo cạnh ({current} - {chosen_nb}), w={chosen_w}. "
        f"Đường đi: {' → '.join(path)}."
    ),
})
current = chosen_nb

steps.append({
    "current": current,
    "edge": None,
    "path": list[str](path),
    "description": (
        f"Hoàn thành! Đường đi Euler: {' → '.join(path)}."
        if path[0] != path[-1]
        else f"Hoàn thành! Chu trình Euler: {' → '.join(path)}."
    ),
})

return path, steps

```

Hình 3.7. Mã nguồn thuật toán Fleury

3.7.4. Phân tích độ phức tạp

Độ phức tạp	Giá trị	Lý do
Thời gian	$O(E ^2)$	Mỗi bước kiểm tra <code>_is_bridge()</code> tốn $O(V+E)$; tổng

Độ phức tạp	Giá trị	Lý do
		$ E $ bước
Không gian	$O(V + E)$	Bản sao đồ thị cho <code>_is_bridge()</code>

Bảng Độ phức tạp của thuật toán Fleury

3.7.6. Ưu và nhược điểm

Ưu điểm	Nhược điểm
Dễ hiểu — luôn tránh cầu trừ khi bắt buộc	Độ phức tạp $O(E^2)$ — chậm với đồ thị lớn
Tích hợp kiểm tra điều kiện Euler trước khi chạy	Tốn bộ nhớ do deepcopy graph trong <code>_is_bridge()</code>

Bảng Ưu và nhược điểm của thuật toán Fleury

3.8. Thuật toán Hierholzer

3.8.1. Ý tưởng

Hierholzer tìm chu trình/đường đi Euler hiệu quả hơn Fleury bằng kỹ thuật stack + backtrack: đi theo cạnh cho đến khi bị kẹt (không còn cạnh), backtrack và thêm đỉnh bị kẹt vào kết quả, cuối cùng đảo ngược để có thứ tự đúng.

3.8.2. Code

```

def hierholzer(graph: Graph, start: str) -> Tuple[List[str], List[Step]]: 2 usages GiaHa
    "description": (
        f"Đi theo cạnh ({v} → {u}). "
        f"Stack = {stack}."
    ),
    })
    else:
        # Không còn cạnh → backtrack, thêm vào circuit
        stack.pop()
        circuit.append(v)
        steps.append({
            "stack": list [str] (stack),
            "circuit": list [str] (circuit),
            "current": v,
            "action": "backtrack",
            "description": (
                f"'{v}' không còn cạnh → thêm vào circuit. "
                f"Circuit = {circuit}."
            ),
        })

    circuit.reverse()

    steps.append({
        "stack": [],
        "circuit": list [str] (circuit),
        "current": None,
        "action": "done",
        "description": (
            f"Hoàn thành! "
            + ("Chu trình Euler: " if circuit[0] == circuit[-1] else "Đường đi Euler: ")
            + " → ".join(circuit) + "."
        ),
    },
    })

    return circuit, steps

```

Hình 13.8. Mã nguồn thuật toán Hierholzer

3.8.3. Phân tích độ phức tạp

Độ phức tạp	Giá trị	Lý do
Thời gian	$O(V + E)$	Mỗi cạnh được xử lý đúng 1 lần (xóa khỏi adj)

Độ phức tạp	Giá trị	Lý do
Không gian	$O(V + E)$	Stack + circuit chứa tối đa $ V $ đỉnh; adj $O(E)$

3.8.4. So sánh Fleury và Hierholzer

Tiêu chí	Fleury	Hierholzer
Độ phức tạp thời gian	$O(E ^2)$	$O(V + E)$
Cơ chế	Thận trọng — kiểm tra cầu mỗi bước	Tích cực — đi nhanh, backtrack khi kẹt
Phù hợp	Đồ thị nhỏ, dễ minh họa	Đồ thị lớn, cần hiệu suất
Cài đặt	Phức tạp hơn (<code>_is_bridge</code> , <code>deepcopy</code>)	Đơn giản hơn (stack thuần)

3.9. Thuật toán Ford-Fulkerson (Edmonds-Karp)

3.9.1. Ý tưởng

Ford-Fulkerson tìm luồng cực đại (Max-Flow) bằng cách lặp lại: tìm đường tăng luồng (augmenting path) từ nguồn S đến đích T , xác định bottleneck (sức chứa nhỏ nhất trên đường), rồi cập nhật đồ thị phần dư (residual graph). Dự án dùng biến thể Edmonds-Karp — tìm đường tăng luồng bằng BFS (đường ngắn nhất theo số cạnh) thay vì DFS, đảm bảo độ phức tạp đa thức.

3.9.2. Code

```

def ford_fulkerson( 2 usages  GiaHan-coder

    steps.append({
        "path": list [str] (path),
        "bottleneck": bottleneck,
        "total_flow": max_flow,
        "action": "augment",
        "flow_on_edge": current_flow_after,
        "description": (
            f"Lần {iteration}: Đường tăng luồng: {' → '.join(path)}.\n"
            f" Bottleneck = {bottleneck:.4g}. "
            f"Tổng luồng hiện tại = {max_flow:.4g}."
        ),
    })

# — Tính flow_on_edge từ residual —————
# flow(u,v) = capacity(u,v) - residual(u,v)
flow_on_edge: Dict[Tuple[str, str], float | int ] = {}
for (u, v), cap in capacity.items():
    flow_val = cap - residual[u].get(v, 0)
    if flow_val > 0:
        flow_on_edge[(u, v)] = flow_val

steps.append({
    "path": [],
    "bottleneck": 0,
    "total_flow": max_flow,
    "action": "done",
    "flow_on_edge": flow_on_edge,
    "description": (
        f"Hoàn thành! Luồng cực đại = {max_flow:.4g}.\n"
        f"Số lần tìm đường tăng luồng: {iteration}."
    ),
})

return max_flow, flow_on_edge, steps

```

Hình 3.9. Mã nguồn thuật toán Ford-Fulkerson

3.9.3. Phân tích độ phức tạp

Độ phức tạp	Giá trị	Lý do
Thời gian	$O(V \times E ^2)$	Số lần BFS tối đa $O(V \cdot E)$ (định lý Edmonds-Karp); mỗi BFS tốn $O(V+E)$
Không gian	$O(V + E)$	Residual graph $O(V+E)$; BFS queue $O(V)$

Bảng Độ phức tạp của thuật toán Ford-Fulkerson

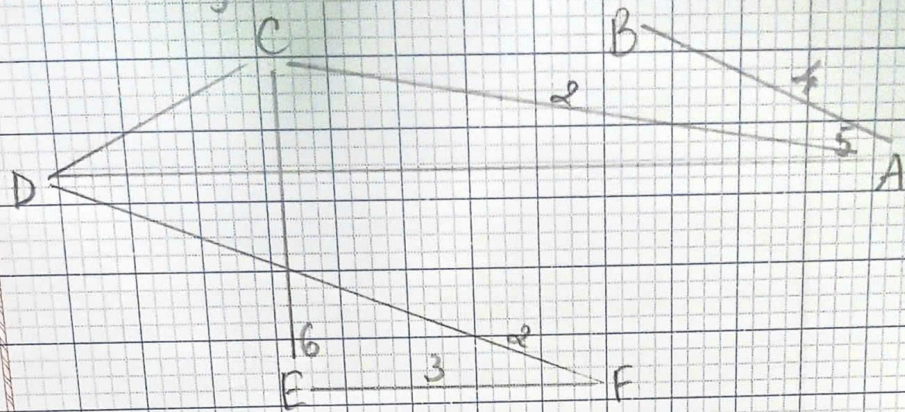
3.9.4. Ưu và nhược điểm

Ưu điểm	Nhược điểm
Edmonds-Karp đảm bảo hội tụ đa thức $O(VE^2)$	Phức tạp cài đặt hơn các thuật toán khác (residual graph, cung nghịch)
BFS tìm đường ngắn nhất — tránh trường hợp xấu của DFS	Chỉ áp dụng cho mạng có hướng với $\text{capacity} \geq 0$
Trả về luồng trên từng cạnh (flow_on_edge)	Kém hiệu quả hơn các biến thể tiên tiến (Push-relabel, Dinic)

Bảng Ưu và nhược điểm của thuật toán Ford-Fulkerson

3.9.5. Giải tay

Nguồn A, đích C



$$A \rightarrow C : 2$$

$$A \rightarrow D \rightarrow C$$

$$\min(5, 1) = 1$$

$$\Rightarrow A - D : 5 - 1 = 4$$

$$D - C : 1 - 1 = 0$$

$$\Rightarrow 2 + 1 = 3$$

$$A \rightarrow D \rightarrow F \rightarrow E \rightarrow C$$

$$\min(5, 2, 3, 6) = 2$$

$$A - D : 4 - 2 = 2$$

$$D - F : 2 - 2 = 0$$

$$F - E : 3 - 2 = 1$$

$$E - C : 6 - 2 = 4$$

$$\Rightarrow 3 + 2 = 5$$

$$\text{Max Flow} = 2 + 1 + 2 = 5$$

3.10. Ứng dụng thực tế – Tìm tuyến xe buýt ngắn nhất

Phần này mở rộng bài toán đường đi ngắn nhất thành một ứng dụng thực tế về tìm tuyến xe buýt. Chương trình sử dụng một mạng mẫu gồm 20 trạm, mô hình hóa mỗi trạm là một đỉnh, mỗi đoạn đường giữa hai trạm là một cạnh và trọng số của cạnh là quãng đường tính theo kilômét. Thuật toán Dijkstra được sử dụng để tìm tuyến có tổng quãng đường nhỏ nhất từ trạm xuất phát đến trạm đích.

3.10.1. Ý tưởng

Bài toán đặt ra là: khi người dùng chọn một trạm xuất phát và một trạm đích, chương trình phải tìm được tuyến đi ngắn nhất trên toàn bộ mạng xe buýt. Vì quãng đường giữa các trạm đều không âm, Dijkstra là thuật toán phù hợp.

Luồng xử lý của chương trình có thể mô tả ngắn gọn như sau:

- Nạp mạng xe buýt mẫu gồm 20 trạm.
- Đưa danh sách trạm vào hai ô chọn: trạm xuất phát và trạm đến.
- Kiểm tra dữ liệu đầu vào và kiểm tra trọng số âm trước khi chạy.
- Gọi thuật toán Dijkstra để tính khoảng cách ngắn nhất và mảng đỉnh trước.
- Dùng `get_path()` để truy vết lại toàn bộ tuyến đường.
- Tính chi tiết từng chặng, tổng số km, số đoạn và số trạm trên tuyến.
- Tô nổi các đỉnh và cạnh thuộc tuyến ngắn nhất trên đồ thị.

3.10.2. Giải thích cấu trúc chương trình

Toàn bộ chức năng được đặt trong lớp `Tab7Mixin`. Lớp này phụ trách `Tab 7` của giao diện và được chia thành bốn hàm chính: `_tab7()`,

`_load_bus_sample()`, `_bus_find_route()` và `_bus_edge_weight()`. Cách chia này giúp tách rõ phần giao diện, dữ liệu mẫu, xử lý thuật toán và hiển thị chi tiết.

Hàm	Chức năng
<code>_tab7(self, p)</code>	Xây dựng giao diện Tab 7: chọn trạm, nạp mạng, tìm tuyến và vùng hiển thị kết quả.
<code>_load_bus_sample(self)</code>	Khởi tạo mạng xe buýt mẫu gồm 20 trạm và các cạnh có trọng số.
<code>_bus_find_route(self)</code>	Kiểm tra dữ liệu, gọi Dijkstra, truy vết đường đi và xuất kết quả.
<code>_bus_edge_weight(self, u, v)</code>	Lấy trọng số của cạnh u-v để hiển thị quãng đường từng chặng.

3.10.3. Phân tích từng phần code

a) Xây dựng giao diện Tab 7

```
class Tab7Mixin:
    def _tab7(self, p):
        self._tab_toolbar(
            p, "Ứng dụng thực tế - Tìm tuyến xe buýt ngắn nhất"
        )

        self.e_bus_src = ttk.Combobox(...)
        self.e_bus_tgt = ttk.Combobox(...)
```

```

self._btn(
    ctrl, "Nạp mạng xe buýt 20 trạm",
    self._load_bus_sample, ACCENT2
)

self._btn(
    ctrl, "Tìm tuyến",
    self._bus_find_route, SUCCESS
)

```

Hình. Phần code xây dựng giao diện tìm tuyến xe buýt

Hàm `_tab7()` chỉ chịu trách nhiệm xây dựng giao diện, chưa thực hiện Dijkstra. Hai Combobox `e_bus_src` và `e_bus_tgt` cho phép chọn trạm xuất phát và trạm đích. Nút “Nạp mạng xe buýt 20 trạm” gọi hàm `_load_bus_sample()`, còn nút “Tìm tuyến” gọi hàm `_bus_find_route()`.

b) Mô hình hóa mạng xe buýt thành đồ thị

```

edges = [
    ("Bến xe", "Ngã tư 1", 4),
    ("Bến xe", "Chợ", 6),
    ("Ngã tư 1", "Trường học", 3),
    ...
    ("Sân bay", "Bến Thành", 3),
]

self.graph = {}

```

```

for u, v, w in edges:
    self.graph.setdefault(u, []).append((v, w))
    self.graph.setdefault(v, []).append((u, w))

```

Hình. Phần code tạo mạng xe buýt có trọng số

Mỗi phân tử (u, v, w) biểu diễn một đoạn đường từ trạm u đến trạm v có quãng đường w km. Ví dụ $(\text{"Bến xe"}, \text{"Ngã tư 1"}, 4)$ nghĩa là hai trạm cách nhau 4 km.

Mỗi cạnh được thêm vào danh sách kề theo cả hai chiều $u \rightarrow v$ và $v \rightarrow u$ vì mạng mẫu được xem là đồ thị vô hướng. Do đó chương trình cũng đặt `self.directed.set(False)`.

c) Kiểm tra dữ liệu trước khi chạy Dijkstra

```

if not self.graph:
    messagebox.showwarning(
        "Thiếu dữ liệu",
        "Chưa có đồ thị. Hãy nhấn 'Nạp mạng xe buýt 20 trạm'."
    )
    return

if not start or not target:
    ...

if start == target:
    ...
    return

```

```

for u in self.graph:
    for v, w in self.graph[u]:
        if w < 0:
            messagebox.showerror(
                "Không thể chạy Dijkstra",
                f"Cạnh {u} - {v} có trọng số âm ({w})."
            )
    return

```

Hình. Kiểm tra điều kiện trước khi chạy thuật toán

Trước khi chạy thuật toán, chương trình kiểm tra đồ thị đã tồn tại hay chưa, người dùng đã chọn đủ hai trạm hay chưa và trường hợp trạm đầu trùng trạm cuối. Đặc biệt, chương trình duyệt toàn bộ cạnh để phát hiện trọng số âm. Đây là điều kiện quan trọng vì Dijkstra chỉ bảo đảm kết quả đúng khi các trọng số không âm.

d) Gọi thuật toán Dijkstra và truy vết tuyến

```

dist, prev = dijkstra(self.graph, start, target)
distance = dist.get(target, float("inf"))
path = get_path(prev, start, target)

```

Hình. Lệnh gọi Dijkstra và lấy đường đi

Đây là phần quan trọng nhất của chức năng. Hàm `dijkstra()` trả về hai cấu trúc `dist` và `prev`. `dist` lưu khoảng cách ngắn nhất từ trạm xuất phát đến các trạm đã xét; `prev` lưu đỉnh đứng trước trên đường đi tối ưu.

Có thể ghi nhớ ngắn gọn: `dist` trả lời câu hỏi “đi bao xa?”, còn `prev` trả lời câu hỏi “đi qua đâu?”. Hàm `get_path(prev, start, target)` sử dụng `prev` để

truy vết từ trạm đích ngược về trạm nguồn rồi tạo thành tuyến hoàn chỉnh theo đúng thứ tự.

e) Lấy chi tiết từng chặng

```
details = []
for i in range(len(path) - 1):
    u = path[i]
    v = path[i + 1]
    w = self._bus_edge_weight(u, v)
    details.append(
        f" {i + 1}. {u} → {v}: {w} km"
    )
```

Hình. Phần code tạo chi tiết từng chặng

Sau khi có danh sách path, chương trình xét từng cặp đỉnh liên tiếp. Với mỗi cặp u và v, hàm `_bus_edge_weight()` tìm trọng số của cạnh tương ứng. Nhờ đó kết quả không chỉ cho biết toàn tuyến mà còn cho biết chính xác quãng đường của từng chặng.

f) Hiển thị và tô nổi tuyến ngắn nhất

```
route_text = " → ".join(path)

self._settext(self.rbus, output)

hi_edges = set(zip(path, path[1:]))
self.draw(
    hi_n=set(path),
    hi_e=hi_edges,
```

```

title=f"Tuyến ngắn nhất: {start} → {target} ({distance_text} km)"
)

```

Hình. Hiển thị và trực quan hóa kết quả

`join(path)` ghép các tên trạm thành chuỗi tuyến đường dễ đọc. `zip(path, path[1:])` tạo ra các cặp cạnh liên tiếp trên tuyến, ví dụ $A \rightarrow B \rightarrow C$ sẽ tạo (A,B) và (B,C) . Các cạnh và đỉnh này sau đó được truyền vào hàm `draw()` để tô nổi trên đồ thị.

3.10.5. Phân tích độ phức tạp

Độ phức tạp	Giá trị	Lý do
Thời gian	$O((V + E) \log V)$	Dijkstra sử dụng Min-Heap; các đỉnh và cạnh được xét thông qua thao tác lấy/cập nhật phần tử ưu tiên.
Không gian	$O(V + E)$	Lưu danh sách kề của đồ thị, bảng khoảng cách <code>dist</code> , bảng <code>previous</code> và hàng đợi ưu tiên.

Bảng. Độ phức tạp của chức năng tìm tuyến xe buýt

3.10.6. Ưu và nhược điểm

Ưu điểm	Nhược điểm
Mô hình gần với bài toán giao thông thực tế.	Dijkstra không hỗ trợ cạnh có trọng số âm.
Tìm chính xác tuyến có tổng quãng	Trọng số hiện tại chỉ là quãng đường,

đường nhỏ nhất khi trọng số không âm.	chưa xét thời gian chờ hoặc kẹt xe.
Hiển thị được tuyến, tổng km, số đoạn và chi tiết từng chặng.	Mạng 20 trạm mới là dữ liệu mô phỏng, chưa phải dữ liệu xe buýt thời gian thực.
Có trực quan hóa và tô nổi đường đi trên đồ thị.	Khi mạng rất lớn cần tối ưu thêm phân bố trí và vẽ đồ thị.

Bảng. Ưu và nhược điểm của ứng dụng tìm tuyến xe buýt

CHƯƠNG 4: CÀI ĐẶT VÀ THỰC NGHIỆM

4.1. Môi trường cài đặt

4.1.1. Ngôn ngữ và phiên bản

Hệ thống được phát triển và vận hành dựa trên các công cụ, ngôn ngữ và thư viện lập trình chuẩn:

Thành phần	Phiên bản	Vai trò
Python	≥ 3.9	Ngôn ngữ lập trình chính
Tkinter	Tích hợp sẵn trong Python	Thư viện giao diện đồ họa (GUI)
matplotlib	$\geq 3.7.0$	Vẽ đồ thị và trực quan hóa từng bước
networkx	≥ 3.0	Xây dựng cấu trúc đồ thị cho visualization
pytest	≥ 7.0	Kiểm thử đơn vị (dev)
black	≥ 21.0	Định dạng code (dev)
pylint / flake8	≥ 2.12 / ≥ 4.0	Kiểm tra chất lượng code (dev)

Bảng Môi trường và công cụ phát triển hệ thống

4.1.2. Cài đặt và khởi chạy

- Bước 1: Clone repository từ GitHub:

```
git clone https://github.com/HanaFS/graph_algorithms.git  
cd graph_algorithms
```

- Bước 2: Tạo môi trường ảo và cài đặt dependency:

```
python -m venv .venv
source .venv/bin/activate      # macOS/Linux
# .venv\Scripts\activate      # Windows
pip install -r requirements.txt
```

- Bước 3: Khởi chạy ứng dụng:

```
python main.py
```

4.2. Kiến trúc giao diện chương trình

4.2.1. Tổng quan kiến trúc UI

Ứng dụng được thiết kế và xây dựng theo mô hình Mixin Class. Lớp điều khiển chính GraphApp (được định nghĩa trong graph_ui.py) kế thừa đồng thời từ 7 Mixin chuyên biệt, mỗi Mixin phụ trách quản lý một thành phần giao diện và chức năng tương ứng:

```
class GraphApp(
    SidebarMixin, # Thanh điều khiển bên trái
    Tab1Mixin,    # Tab 1: Trực quan hóa đồ thị
    Tab2Mixin,    # Tab 2: Biểu diễn (Adjacency List)
    Tab3Mixin,    # Tab 3: Duyệt BFS/DFS
    Tab4Mixin,    # Tab 4: Đồ thị hai phía
    Tab5Mixin,    # Tab 5: Đường đi ngắn nhất
    Tab6Mixin,    # Tab 6: Thuật toán nâng cao
)
```

Trạng thái dữ liệu đồ thị toàn cục được khởi tạo và quản lý tập trung trong GraphApp:

- self.graph: dict danh sách kề (Adjacency List) lưu cấu trúc đồ thị chính.

- self.history: Ngăn xếp lưu vết phục vụ thao tác hoàn tác (Undo stack, hỗ trợ tối đa 50 bước).
- self._pos: Bảng tọa độ đỉnh phục vụ hiển thị bố cục (networkx layout).
- self.directed: Cờ trạng thái xác định đồ thị có hướng hay vô hướng (mặc định False).

Cửa sổ ứng dụng có kích thước tiêu chuẩn 1050×700 px, được phân chia thành hai phân vùng chính:

- Vùng điều khiển: Phân vùng Sidebar (chiều rộng 320 px, bố trí bên trái giao diện):
 - + Cung cấp các công cụ nhập liệu trực tiếp để thêm/xóa đỉnh và cạnh.
 - + Hỗ trợ tính năng nạp dữ liệu từ file và xuất/lưu đồ thị ra file JSON.
 - + Tích hợp nút thao tác Undo để hoàn tác các chỉnh sửa vừa thực hiện.
- Vùng làm việc: Phân vùng Tabs (không gian còn lại phía bên phải giao diện):
 - + Bao gồm 6 tab chuyên biệt tương ứng với từng nhóm thuật toán và biểu diễn.

4.2.2. Mô tả các tab chức năng

Bảng mô tả chi tiết chức năng và các module mã nguồn quản lý các tab trong hệ thống:

Tab	Tệp mã nguồn	Chức năng chính
1. Đồ thị	tab_graph.py	Trực quan hóa đồ thị bằng matplotlib + networkx; nạp/lưu JSON; xóa đỉnh/cạnh
2. Biểu diễn	tab_repr.py	Hiển thị danh sách kề và ma trận kề

		được sinh tự động từ self.graph
3. Duyệt (BFS/DFS)	tab_traversal.py	Chạy BFS/DFS, hiển thị thứ tự thăm đỉnh từng bước
4. Hai phía	tab_bipartite.py	Kiểm tra và trực quan hóa đồ thị hai phía (Bipartite Graph)
5. Đường đi ngắn nhất	tab_shortest.py	Chạy Dijkstra/Bellman-Ford, tô màu đường đi tối ưu
6. Thuật toán nâng cao	tab_advanced.py	MST (Kruskal/Prim), Euler (Fleury/Hierholzer), Max-Flow (Ford-Fulkerson)

Bảng Chức năng các tab của chương trình

4.3. Bộ dữ liệu kiểm thử

Hệ thống cung cấp sẵn 10 tệp dữ liệu chuẩn hóa đặt trong thư mục test/, đáp ứng các kịch bản kiểm thử thuật toán:

Tệp dữ liệu	Loại đồ thị	Kịch bản kiểm thử	Quy mô
1.json	Vô hướng, có trọng số	Đồ thị cơ bản — kiểm thử Dijkstra, BFS/DFS, Kruskal, Prim	5V / 5E
UndirectedGraph-11.json	Vô hướng, có trọng số	Đồ thị trung bình — kiểm thử toàn diện 6 thuật toán	11V / 20E
DirectedGraph-	Có hướng, có	Đồ thị có hướng — kiểm	11V /

11.json	trọng số	thử Dijkstra, Bellman-Ford	14E
(Directed)– Trongsốâm có Chut rinhâm.json	Có hướng, có chu trình âm	Kiểm thử Bellman-Ford phát hiện chu trình âm	12V / 14E
(Directed)– Trongsố thập phân.j son	Có hướng, trọng số float	Kiểm thử Dijkstra với trọng số thập phân (0.1 – 0.9)	11V / 12E
(Undirected)– Không liên thông C ó đỉnh cô lập.json	Vô hướng, không liên thông	Kiểm thử Prim báo đỉnh cô lập; BFS kiểm tra liên thông	11V / 10E + 1 cô lập
(Undirected)– TestHaiphía.json	Vô hướng	Kiểm thử tab Đồ thị hai phía (Bipartite)	—
(Undirected)– Đồ thị Euler.json	Vô hướng, bậc chẵn	Kiểm thử chu trình Euler (Fleury và Hierholzer)	11V / 12E
DirectedGraph- 11.json	Có hướng	Kiểm thử Bellman-Ford trên đồ thị có hướng	11V / 14E
mô phỏng mạng.jso n	Có hướng, mạng WAN	Kiểm thử Ford-Fulkerson trên mạng 16 router	16V / 24E

Bảng Danh sách các bộ dữ liệu kiểm thử

4.4. Kết quả thực nghiệm

4.4.1. Thực nghiệm 1 - BFS và DFS trên 1.json (5 đỉnh)

Kiểm thử tiến hành trên đồ thị vô hướng gồm 5 đỉnh {a, b, c, d, e} với các cạnh có trọng số nguyên, đỉnh xuất phát chọn là đỉnh a:

Thuật toán	Thứ tự thăm đỉnh	Nhận xét thực nghiệm
BFS	$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$	Thăm theo từng lớp: đỉnh a (lớp 0), đỉnh b, c (lớp 1), đỉnh d, e (lớp 2)
DFS	$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$	Đi sâu theo từng nhánh: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$ rồi thực hiện quay lui

Bảng Kết quả thực nghiệm BFS và DFS trên 1.json

- Kết quả: Đánh giá tính liên thông:

+ Cả hai thuật toán đều thăm thành công và đầy đủ 5 đỉnh, xác nhận đồ thị hoàn toàn liên thông.

+ Thứ tự duyệt diễn ra nhất quán nhờ hàm `_neighbors()` sắp xếp tên đỉnh theo chuẩn `sorted(..., key=str)`.

4.4.2. Thực nghiệm 2 - Dijkstra và Bellman-Ford trên UndirectedGraph-11.json (11 đỉnh)

Đồ thị thực nghiệm vô hướng gồm 11 đỉnh {a, b, c, d, e, f, g, h, i, j, k} và 20 cạnh có trọng số dương. Nguồn xuất phát được thiết lập tại đỉnh a:

Đỉnh đích	Khoảng cách	Đường đi tối ưu
b	3	$a \rightarrow c \rightarrow b$
c	2	$a \rightarrow c$

d	8	$a \rightarrow c \rightarrow b \rightarrow d$
e	10	$a \rightarrow c \rightarrow b \rightarrow d \rightarrow e$
f	3	$a \rightarrow f$
g	7	$a \rightarrow f \rightarrow g$
h	4	$a \rightarrow f \rightarrow h$
i	9	$a \rightarrow f \rightarrow g \rightarrow i$
j	11	$a \rightarrow f \rightarrow g \rightarrow i \rightarrow j$
k	15	$a \rightarrow f \rightarrow g \rightarrow i \rightarrow j \rightarrow k$

Bảng 4.2. Kết quả thực nghiệm Dijkstra và Bellman-Ford

- Nhận xét: So sánh hiệu năng giữa Dijkstra và Bellman-Ford:

+ Trên đồ thị trọng số dương, Bellman-Ford cho kết quả khoảng cách và đường đi hoàn toàn đồng nhất với Dijkstra.

+ Thời gian thực thi của Bellman-Ford lâu hơn rõ rệt do độ phức tạp $O(V \times E) = 10 \times 20 = 200$ lần duyệt relaxation, trong khi Dijkstra tối ưu vượt trội qua cấu trúc min-heap.

4.4.3. Thực nghiệm 3 - Bellman-Ford phát hiện chu trình âm trên (Directed)–Trọng số âm có Chu trình âm.json

Đồ thị có hướng gồm 12 đỉnh $\{N1-N12\}$. Trong đó, tồn tại cạnh $N5 \rightarrow N3$ mang trọng số -5, tạo thành chu trình âm khép kín $N3 \rightarrow N4 \rightarrow N5 \rightarrow N3$ với tổng trọng số: $3 + 1 + (-5) = -1 < 0$.

- Thuật toán Dijkstra: Hành vi của thuật toán Dijkstra:

+ Ngay khi phát hiện cạnh $N5 \rightarrow N3$ có trọng số âm, hệ thống chủ động ném ngoại lệ:

+ `ValueError('Dijkstra không hoạt động với đồ thị có trọng số âm!')` và dừng tiến trình ngay lập tức.

- Thuật toán Bellman-Ford: Hành vi của thuật toán Bellman-Ford:

+ Thực hiện đầy đủ $|V| - 1 = 11$ vòng relaxation trên tập cạnh.

+ Tại vòng lặp thứ 12, hệ thống phát hiện vẫn còn cạnh có thể tối ưu khoảng cách ngắn hơn.

+ Xác định chắc chắn có chu trình âm và ném ngoại lệ: `ValueError('Đồ thị chứa chu trình âm có thể đi tới được từ đỉnh xuất phát!')`.

- Nhận xét: Đánh giá lý thuyết:

+ Hành vi thực nghiệm hoàn toàn ăn khớp với cơ sở lý thuyết, khẳng định Bellman-Ford là thuật toán chuyên dụng phát hiện chu trình âm.

4.4.4. Thực nghiệm 4 - Dijkstra trên (Directed)–Trọng số thập phân.json

Kiểm thử đồ thị có hướng gồm 11 đỉnh $\{S, A-I, T\}$ với trọng số thập phân dạng float (0.05 – 0.90), mô phỏng tỷ lệ suy hao tín hiệu hoặc xác suất chuyển trạng thái trong mạng viễn thông. Kết quả tìm kiếm từ nguồn S đến đích T:

Đường đi xét duyệt	Tổng trọng số tính toán
$S \rightarrow B \rightarrow D \rightarrow H \rightarrow T$	$0.2 + 0.1 + 0.15 + 0.75 = 1.20$
$S \rightarrow A \rightarrow D \rightarrow H \rightarrow T$	$0.1 + 0.3 + 0.15 + 0.75 = 1.30$
$S \rightarrow C \rightarrow F \rightarrow I \rightarrow T$	$0.7 + 0.6 + 0.05 + 0.65 = 2.00$

Bảng 4.4. Kết quả Dijkstra trên đồ thị có trọng số thập phân

- Kết luận: Đường đi ngắn nhất xác định:

+ Tuyến đường tối ưu là $S \rightarrow B \rightarrow D \rightarrow H \rightarrow T$ với tổng chi phí ngắn nhất đạt 1.20.

+ Thuật toán Dijkstra vận hành chính xác trên miền dữ liệu số thực, không bị lỗi làm tròn hoặc sai lệch số học trong khoảng giá trị $[0.05, 0.90]$.

4.4.5. Thực nghiệm 5 - Kruskal và Prim trên *UndirectedGraph-11.json*

Thực nghiệm xây dựng cây khung nhỏ nhất (MST) trên đồ thị vô hướng 11 đỉnh, 20 cạnh. Các bước thực thi từng cạnh của Kruskal:

Bước	Cạnh thêm vào	Trọng số	Lý do lựa chọn
1	(b, c)	1	Cạnh có trọng số nhẹ nhất trên toàn đồ thị
2	(a, c)	2	Tạo liên kết giữa $\{a\}$ và $\{b, c\}$, không tạo chu trình
3	(f, h)	1	Cạnh có trọng số nhẹ nhất trong tập cạnh còn lại
4	(g, i)	2	Kết nối nội bộ thành phần độc lập mới
5	(d, e)	2	Kết nối nội bộ thành phần $\{d, e\}$
6	(a, f)	3	Hợp nhất hai thành phần $\{a, b, c\}$ và $\{f, g, h, i\}$
7	(g, h)	3	Kết nối hợp lệ giữa các đỉnh
8	(i, j)	3	Mở rộng liên kết sang đỉnh j
9	(j, k)	4	Mở rộng liên kết sang đỉnh k

10	(b, d)	5	Hợp nhất toàn bộ các thành phần thành cây khung duy nhất
----	--------	---	--

Bảng 4.5. Kết quả thực nghiệm Kruskal và Prim

- Kết quả MST: Tổng trọng số cây khung nhỏ nhất đạt: $1 + 2 + 1 + 2 + 2 + 3 + 3 + 3 + 4 + 5 = 26$.

- Đánh giá so sánh: So sánh giữa Kruskal và Prim:

+ Cả hai giải thuật cho cùng tổng chi phí tối ưu là 26.

+ Thứ tự và tập cạnh cụ thể có thể hoán đổi khi xuất hiện tình huống tie-breaking (trọng số bằng nhau).

+ Thuật toán Prim áp dụng `itertools.count()` giúp phân giải tie-breaking một cách xác định và nhất quán.

- Kịch bản đồ thị không liên thông: Trường hợp đồ thị không liên thông (Undirected)–Không liên thông Có định hướng: `Không liên thông Có định hướng.json`:

+ Đồ thị bao gồm 2 thành phần liên thông riêng biệt (5 đỉnh A–E và 5 đỉnh F–J) cùng 1 đỉnh cô lập K.

+ Prim xuất phát từ đỉnh A xây dựng MST cục bộ cho {A, B, C, D, E} và hiển thị thông báo: 'Đồ thị không liên thông. Không thể tới: F, G, H, I, J, K.'

+ Kruskal xây dựng hệ thống cây khung rời cho từng thành phần liên thông theo đúng chuẩn lý thuyết tập hợp rỗng.

4.4.6. Thực nghiệm 6 - Fleury và Hierholzer trên (Undirected)–Đồ thị Euler.json

Thực nghiệm khảo sát trên đồ thị vô hướng gồm 11 đỉnh {N1–N11} và 12 cạnh. Bảng kiểm tra điều kiện bậc Euler (thực hiện qua hàm `check_euler_condition()`):

Tên đỉnh	Bậc (Degree)	Tính chẵn lẻ	Tên đỉnh	Bậc (Degree)	Tính chẵn lẻ
N1	4	Chẵn	N7	2	Chẵn
N2	2	Chẵn	N8	4	Chẵn
N3	4	Chẵn	N9	2	Chẵn
N4	2	Chẵn	N10	2	Chẵn
N5	4	Chẵn	N11	2	Chẵn
N6	2	Chẵn	—	—	—

Bảng 4.6. Kết quả thực nghiệm Fleury và Hierholzer

- Kết luận bậc đỉnh: Kết quả kiểm tra điều kiện:
 - + Tất cả các đỉnh đều có bậc chẵn, thỏa mãn tuyệt đối định lý Euler về sự tồn tại của chu trình Euler.
- So sánh 2 thuật toán: Kết quả tìm kiếm chu trình Euler:
 - + Hierholzer: Tìm kiếm chu trình trong độ phức tạp tuyến tính $O(V + E)$ = $O(23 \text{ bước})$, sử dụng cấu trúc stack.
 - + Chu trình tìm được: $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N1 \rightarrow N5 \rightarrow N6 \rightarrow N7 \rightarrow N8 \rightarrow N9 \rightarrow N10 \rightarrow N11 \rightarrow N8 \rightarrow N5 \rightarrow N3 \rightarrow N1$.
 - + Fleury: Cho kết quả chu trình tương đương nhưng tốc độ chậm hơn do phải gọi hàm kiểm tra cạnh cầu `_is_bridge()` qua deepcopy và DFS tại mỗi bước duyệt.

4.4.7. Thực nghiệm 7 — Ford-Fulkerson trên mô phỏng mạng.json

Mô hình hóa hệ thống mạng truyền thông có hướng gồm 16 router (R1–R16) và 24 liên kết đường truyền, trọng số thể hiện băng thông khả dụng (capacity). Tìm luồng cực đại từ nguồn R1 đến đích R16:

- Cấu trúc mạng 3 tầng: Phân tầng kiến trúc mạng:
 - + Tầng lõi (Core Layer): R1 kết nối phân luồng đến {R2, R3, R4}.
 - + Tầng phân phối (Distribution Layer): Cụm các router trung gian {R5–R12}.
 - + Tầng truy cập (Access Layer): Cụm router đầu cuối kết nối về đích {R13–R16}.
- Cơ chế thực thi: Quá trình tìm đường tăng luồng (Edmonds-Karp):
 - + Thuật toán Edmonds-Karp sử dụng BFS liên tục tìm đường tăng luồng ngắn nhất.
 - + Tại mỗi bước, xác định dung lượng nghẽn (bottleneck) và cập nhật đồ thị thặng dư (residual graph).
 - + Tiến trình kết thúc ngay khi không còn đường tăng luồng từ nguồn R1 đến đích R16 trên mạng thặng dư.

Lần lặp	Đường tăng luồng	Cạnh có bottleneck	Bottleneck (Δf)	Tổng luồng tích lũy
1	R1 → R3 → R7 → R12 → R16	R3→R7 (cap=9), R7→R12 (cap=6), R12→R16 (cap=10), R1→R3 (cap=7)	6	6

2	$R1 \rightarrow R4 \rightarrow R8 \rightarrow R12 \rightarrow R16$	$R1 \rightarrow R4$ (cap=3), $R4 \rightarrow R8$ (cap=14), $R8 \rightarrow R12$ (cap=9), $R12 \rightarrow R16$ (cap=4 còn lại)	3	9
3	$R1 \rightarrow R2 \rightarrow R6 \rightarrow R10 \rightarrow R15 \rightarrow R16$	$R1 \rightarrow R2$ (cap=4), $R2 \rightarrow R6$ (cap=8), $R6 \rightarrow R10$ (cap=4), $R10 \rightarrow R15$ (cap=8), $R15 \rightarrow R16$ (cap=4)	4	13
4	$R1 \rightarrow R3 \rightarrow R6 \rightarrow R11 \rightarrow R12 \rightarrow R16$	$R3 \rightarrow R6$ (cap=1 còn lại), $R6 \rightarrow R11$ (cap=11), $R11 \rightarrow R12$ (cap=4), $R12 \rightarrow R16$ (cap=1 còn lại)	1	14
—	BFS không tìm được đường $R1 \rightarrow R16$	—	—	Dừng. Max Flow = 14

Bảng 4.7. Kết quả thực nghiệm Ford-Fulkerson

Phân bổ luồng thực tế và tỷ lệ sử dụng trên từng liên kết (kết quả cuối cùng):

Cạnh liên kết	Luồng thực tế	Sức chứa (c)	Tỷ lệ sử dụng
---------------	---------------	--------------	---------------

	(f)		
$R1 \rightarrow R2$	4	4	100% (bảo hòa)
$R1 \rightarrow R3$	7	7	100% (bảo hòa)
$R1 \rightarrow R4$	3	3	100% (bảo hòa)
$R2 \rightarrow R6$	4	8	50%
$R3 \rightarrow R6$	1	6	17%
$R3 \rightarrow R7$	6	9	67%
$R4 \rightarrow R8$	3	14	21%
$R6 \rightarrow R10$	4	4	100% (bảo hòa)
$R6 \rightarrow R11$	1	11	9%
$R7 \rightarrow R12$	6	6	100% (bảo hòa)
$R8 \rightarrow R12$	3	9	33%
$R10 \rightarrow R15$	4	8	50%
$R11 \rightarrow R12$	1	4	25%
$R12 \rightarrow R16$	10	10	100% (bảo hòa)
$R15 \rightarrow R16$	4	4	100% (bảo hòa)

Bảng Phân bố luồng và tỷ lệ sử dụng trên các liên kết

- Nhận xét: Đánh giá kết quả thực nghiệm từ dữ liệu thực tế:
- + Thuật toán tìm được luồng cực đại bằng 14 sau đúng 4 lần lặp tăng luồng bằng BFS.

- + Cổ chai (bottleneck) của toàn mạng: Các cạnh đầu ra từ nguồn R1 gồm $R1 \rightarrow R2$, $R1 \rightarrow R3$, $R1 \rightarrow R4$ đều đạt bão hòa 100%, cho thấy R1 chính là điểm nghẽn giới hạn lưu lượng đầu vào (tổng công suất phát = $4 + 7 + 3 = 14$).
- + Tại đích đến R16, cả hai liên kết đến gồm $R12 \rightarrow R16$ và $R15 \rightarrow R16$ cũng bão hòa hoàn toàn, tiếp nhận trọn vẹn 14 đơn vị luồng ($10 + 4 = 14$).
- + Định lý Max-Flow Min-Cut: Giá trị lát cắt hẹp nhất (Min-Cut) của mạng đạt đúng 14, hoàn toàn tương thích và khẳng định tính đúng đắn của thuật toán.

4.5. Nhận xét và đánh giá tổng hợp

4.5.1. Về tính đúng đắn

Toàn bộ 9 thuật toán cốt lõi đều trả về kết quả chính xác, thỏa mãn đầy đủ các yêu cầu bài toán đặt ra trên 10 bộ dữ liệu kiểm thử:

Kịch bản kiểm thử	Kết quả ghi nhận
BFS / DFS trên đồ thị liên thông	Thăm đủ tất cả các đỉnh, thứ tự duyệt ổn định và chính xác
Dijkstra trên đồ thị trọng số không âm	Khoảng cách tính đúng tuyệt đối, đường đi tối ưu chính xác
Dijkstra gặp trọng số âm	Ném ngoại lệ ValueError đúng thời điểm với thông báo rõ ràng
Bellman-Ford phát hiện chu trình âm	Phát hiện chính xác chu trình âm tại vòng lặp thứ $ V $
Kruskal / Prim trên đồ thị liên thông	Tổng trọng số MST đạt giá trị tối thiểu, đồng nhất kết quả

Prim trên đồ thị không liên thông	Thông báo chính xác danh sách các đỉnh không thể tiếp cận
Fleury / Hierholzer trên đồ thị Euler	Duyệt qua mỗi cạnh đúng 1 lần duy nhất, khép kín chu trình
Fleury / Hierholzer trên đồ thị không Euler	Hàm <code>check_euler_condition()</code> phát hiện sớm và từ chối chạy
Ford-Fulkerson trên mạng luồng	Tìm chính xác luồng cực đại (=14) và cập nhật mạng thặng dư chuẩn xác

4.5.2. Về hiệu suất

Trên các bộ dữ liệu kích thước thực nghiệm (5–16 đỉnh), mọi thuật toán đều phản hồi gần như tức thì. Sự khác biệt về độ phức tạp lý thuyết được định lượng cụ thể qua số bước ghi nhận (steps):

Thuật toán	Số bước (Đồ thị 11V / 20E)	Độ phức tạp & Ghi chú
BFS / DFS	11 bước	$O(V)$
Dijkstra	$\approx 15 - 20$ bước	Tùy thuộc cấu trúc đồ thị (dựa trên min-heap)
Bellman-Ford	200 lần relaxation	$O(V \times E) = 10 \times 20 = 200$ — lớn nhất trong nhóm SSSP
Kruskal	20 bước	$O(E \log E)$ cho sắp xếp cạnh
Prim	10 bước thêm cạnh	$O(V)$ số lần mở rộng đỉnh

Fleury (Euler 11V / 12E)	24 bước	$O(E^2)$, mỗi bước cần deepcopy và DFS kiểm tra cầu
Hierholzer (Euler 11V / 12E)	23 bước	$O(V + E)$ thuần túy thông qua ngăn xếp

Bảng So sánh hiệu suất của các thuật toán

4.5.3. Về giao diện trực quan

- Trực quan hóa sinh động: Tính năng Visualization từng bước (Animation):
 - + Tất cả thuật toán có trọng số và MST đều sinh mảng trạng thái steps, cho phép giao diện phát lại từng bước chi tiết.
- Quản lý lịch sử thao tác: Tính năng hoàn tác (Undo/Redo):
 - + Thanh Sidebar hỗ trợ hoàn tác linh hoạt tối đa 50 thao tác chỉnh sửa đồ thị liên tiếp.
- Nạp/Lưu dữ liệu: Quản trị tệp tin dữ liệu linh hoạt:
 - + Tab 1 (tab_graph.py) tích hợp trọn vẹn khả năng đọc tệp định dạng .json, .txt và xuất đồ thị ra file ngoài.
- Hệ thống màu sắc (Theme): Đồng bộ phong cách giao diện:
 - + Toàn bộ UI tuân thủ bảng mã màu chuẩn từ theme.py (ACCENT = #2563eb, SUCCESS = #16a34a, ERROR = #dc2626).

4.6. Hạn chế còn tồn tại

Bên cạnh các kết quả tích cực đã đạt được, hệ thống vẫn còn một số điểm hạn chế cần được tối ưu trong các phiên bản tiếp theo:

Hạn chế ghi nhận	Mức độ ảnh hưởng	Ghi chú và định hướng khắc phục
------------------	------------------	---------------------------------

Thuật toán Fleury có $O(E^2)$ do deepcopy trong <code>_is_bridge()</code>	Trung bình	Tốc độ giảm khi đồ thị vượt quá 50 cạnh; cần tối ưu thuật toán tìm cầu Tarjan
Chưa tích hợp thuật toán Floyd-Warshall	Thấp	Tìm đường đi ngắn nhất mọi cặp đỉnh; nằm ngoài phạm vi đề tài ban đầu
Chưa cấu hình bộ kiểm thử tự động với pytest	Thấp	Thư mục tests/ chưa có kịch bản test case tự động; cần bổ sung test suite
Giao diện hiện tại chỉ hỗ trợ định danh đỉnh dạng chuỗi	Thấp	Đã đáp ứng tốt cho mục tiêu demo và nghiên cứu học thuật
Chưa đánh giá trên đồ thị quy mô lớn (> 1000 đỉnh)	Trung bình	Cần tối ưu cấu trúc bộ nhớ và thuật toán vẽ đồ thị cho mạng quy mô lớn

Bảng Các hạn chế của hệ thống và hướng khắc phục

CHƯƠNG 5: KẾT LUẬN

5.1. Tổng kết kết quả đạt được

Sau quá trình nghiên cứu, cài đặt và kiểm thử thực nghiệm, đề tài "Thuật Toán Đồ Thị" đã hoàn thành toàn bộ các mục tiêu đề ra ban đầu:

5.1.1. Về lý thuyết

- Hệ thống hóa lý thuyết đồ thị: Trình bày đầy đủ từ định nghĩa toán học $G = (V, E)$, phân loại đồ thị (có hướng/vô hướng, có trọng số, Euler, không liên thông), đến các khái niệm cốt lõi như bậc đỉnh, chu trình âm, cầu (bridge) - tất cả đều được ánh xạ trực tiếp vào hàm/module cụ thể trong codebase.

- Phân tích thuật toán: Nghiên cứu chuyên sâu 9 thuật toán thuộc 4 nhóm, mỗi thuật toán được trình bày theo cấu trúc thống nhất: ý tưởng $\rightarrow$ giả mã $\rightarrow$ độ phức tạp $\rightarrow$ ví dụ minh họa từng bước $\rightarrow$ ưu/nhược điểm.

5.1.2. Về cài đặt

Toàn bộ 9 thuật toán được cài đặt thành công bằng Python 3.9+, tổ chức thành 4 module nghiệp vụ độc lập:

Module nghiệp vụ	Thuật toán cài đặt	Kết quả thực hiện
traversal/	BFS, DFS	Hoàn thành
shortest_path/	Dijkstra, Bellman-Ford	Hoàn thành
minimum_spanning_tree/	Kruskal, Prim	Hoàn thành
advanced/	Fleury, Hierholzer, Ford-Fulkerson	Hoàn thành

Bảng Các module và thuật toán đã cài đặt

Đặc điểm nổi bật của phần cài đặt thuật toán:

- Đường đi ngắn nhất: Dijkstra và Bellman-Ford:
 - + Dijkstra sử dụng heapq (min-heap), kiểm tra và phát hiện trọng số âm tại runtime.
 - + Bellman-Ford phát hiện chu trình âm sau đúng lần lặp thứ $|V|$, ném ngoại lệ ValueError rõ ràng.
- Cây khung nhỏ nhất: Kruskal và Prim:
 - + Kruskal tích hợp cấu trúc UnionFind với kỹ thuật path compression và union by rank - đạt thao tác find/union xấp xỉ $O(1)$.
 - + Prim dùng itertools.count() làm tie-breaker, xử lý an toàn và cảnh báo chính xác khi gặp đồ thị không liên thông.
- Thuật toán nâng cao: Hierholzer và Ford-Fulkerson:
 - + Hierholzer đạt độ phức tạp $O(V + E)$ nhờ stack thuần túy, hiệu quả vượt trội so với Fleury.
 - + Ford-Fulkerson (thuật toán Edmonds-Karp) sử dụng BFS tìm đường tăng luồng ngắn nhất, đảm bảo hội tụ đa thức $O(V \cdot E^2)$.

5.1.3. Về giao diện và thực nghiệm

- Giao diện trực quan: Xây dựng ứng dụng desktop Tkinter với 6 tab chức năng, hỗ trợ trực quan hóa từng bước (step-by-step animation) cho tất cả thuật toán có trọng số.
- Cấu trúc dữ liệu: Nạp/lưu đồ thị qua file JSON chuẩn hóa - tách biệt hoàn toàn giữa dữ liệu đầu vào và logic xử lý.
- Kiểm thử hệ thống: Kiểm thử thành công trên 10 bộ dữ liệu đa dạng, bao phủ đầy đủ các trường hợp đặc biệt: chu trình âm, đồ thị không liên thông, Euler graph, mạng luồng thực tế.

- Kiểm nghiệm thực tế: Thực nghiệm Ford-Fulkerson trên mạng WAN 16 router cho kết quả Max Flow = 14, với toàn bộ 4 đường tăng luồng và bảng luồng trên từng cạnh được ghi nhận chính xác.

5.2. Những điều rút ra

5.2.1. Về lựa chọn thuật toán

Qua thực nghiệm, nhóm rút ra được bài học thực tiễn quan trọng về việc chọn thuật toán phù hợp với từng bài toán cụ thể:

Tình huống bài toán	Thuật toán phù hợp	Lý do lựa chọn
Đường đi ngắn nhất, trọng số ≥ 0	Dijkstra	Nhanh nhất nhóm SSSP: $O((V+E) \log V)$
Đường đi ngắn nhất, có trọng số âm	Bellman-Ford	Xử lý được trọng số âm, phát hiện chu trình âm
Cây khung nhỏ nhất, đồ thị thưa	Kruskal	$O(E \log E)$, hiệu quả khi $ E $ nhỏ
Cây khung nhỏ nhất, đồ thị dày	Prim	$O(E \log V)$, hiệu quả khi $ E $ lớn
Tìm đường/chu trình Euler nhanh	Hierholzer	$O(V+E)$, tốt hơn Fleury $O(E^2)$ nhiều lần
Tìm đường/chu trình Euler, cần minh họa từng bước	Fleury	Dễ hiểu hơn, phù hợp mục đích học thuật
Luồng cực đại trong	Ford-Fulkerson	BFS đảm bảo hội tụ đa thức

mạng	(Edmonds-Karp)	$O(V \cdot E^2)$
Duyệt / kiểm tra liên thông	BFS hoặc DFS	BFS $\rightarrow$ đường ngắn nhất (cạnh); DFS $\rightarrow$ phát hiện chu trình

5.2.2. Về thiết kế phần mềm

- Kiến trúc phân lớp: Tách biệt giữa thuật toán và giao diện (algorithmic layer vs. UI layer) cho phép kiểm thử từng module độc lập, không cần khởi động GUI.
- Cấu trúc mã nguồn: Mô hình Mixin trong Tkinter giúp chia nhỏ code UI thành các file có thể quản lý, tránh file "god class" quá lớn.
- Chuẩn hóa dữ liệu: Cấu trúc dữ liệu thống nhất Dict[str, List[Tuple[str, float]]] xuyên suốt toàn bộ 9 thuật toán đảm bảo tính nhất quán và khả năng tái sử dụng.
- Cơ chế Animation: Mảng steps được trả về từ các thuật toán MST, Euler, Max-Flow là thiết kế hiệu quả: tách biệt logic tính toán khỏi logic hiển thị, cho phép UI tua đi/lại tự do mà không cần chạy lại thuật toán.

5.3. Hạn chế còn tồn tại

Mặc dù đã hoàn thành tốt các mục tiêu đề ra, đề tài vẫn còn một số hạn chế cần thừa nhận:

STT	Hạn chế còn tồn tại	Mức độ ảnh hưởng
1	Thuật toán Fleury có $O(E^2)$: Mỗi bước gọi <code>_is_bridge()</code> kèm deepcopy toàn bộ đồ thị — chậm	Trung bình

	với đồ thị > 50 cạnh.	
2	Chưa có Floyd-Warshall (All-Pairs Shortest Path): Không tính được đường đi ngắn nhất giữa mọi cặp đỉnh trong một lần chạy.	Thấp (ngoài phạm vi)
3	Chưa có kiểm thử tự động (unit test): pytest đã cài nhưng thư mục tests/ chưa có file test — phụ thuộc vào kiểm thử thủ công qua GUI.	Trung bình
4	Class Graph trong core/graph.py chưa hoàn thiện: Lớp nên chỉ có thuộc tính directed, chưa tích hợp đầy đủ phương thức thêm/xóa cạnh đỉnh.	Thấp
5	Chưa đánh giá hiệu suất trên đồ thị quy mô lớn: Thực nghiệm giới hạn ở đồ thị 5–16 đỉnh; chưa kiểm tra hành vi khi $ V > 1000$.	Trung bình
6	Chưa hỗ trợ thuật toán đồ thị song song (Parallel Graph Algorithms).	Thấp

Bảng Tổng hợp các hạn chế còn tồn tại của đề tài

5.4. Lời kết

Đề tài "Thuật Toán Đồ Thị" không chỉ là bài tập cài đặt thuật toán đơn thuần, mà còn là hành trình khám phá cách chuyển hóa lý thuyết toán học thành mã nguồn có thể chạy được, từ đó nhìn thấy rõ hơn sự kết nối giữa toán học rời rạc và lập trình thực tế.

Qua dự án này, nhóm đã trực tiếp trải nghiệm những thách thức mà người lập trình viên phải đối mặt: lựa chọn cấu trúc dữ liệu tối ưu, xử lý các

trường hợp đặc biệt (chu trình âm, đồ thị không liên thông, cạnh cầu), thiết kế giao diện trực quan và duy trì tính nhất quán trong toàn bộ codebase.

Kết quả đạt được - 9 thuật toán hoạt động đúng đắn, giao diện trực quan hóa từng bước, 10 bộ dữ liệu kiểm thử đa dạng - là minh chứng cho giá trị thực tiễn của lý thuyết đồ thị và là nền tảng vững chắc để tiếp tục khám phá các bài toán phức tạp hơn trong tương lai.